

Guia Completo de Estrutura dos Notebooks

Pipeline de Processamento de Biossinais — Dataset PTB-XL

Processamento para Reconhecimento de Padrões em ECG Clínico

Como Usar Este Guia

Este documento é o roteiro completo e autossuficiente para a construção dos notebooks do pipeline. Ele descreve **o que fazer, como fazer, quais parâmetros usar, o que validar e o que salvar** em cada etapa. Seguindo este guia integralmente, o resultado final será um dataset pronto para alimentar algoritmos de Reconhecimento de Padrões, acompanhado de documentação técnica de nível TCC.

Organização: São 10 notebooks numerados (01_ a 10_) mais um entregável final em LaTeX. Cada notebook é independente em execução, mas dependente em dados — os arquivos salvos por um notebook são a entrada do seguinte. A convenção de nomes de arquivo deve ser rigorosamente seguida para garantir rastreabilidade.

Sobre o Dataset PTB-XL: É um dataset público de ECG clínico de 12 derivações composto por 21.837 registros de 10 segundos provenientes de 18.885 pacientes, coletados entre 1989 e 1996 na Alemanha. Os sinais estão disponíveis em dois formatos de frequência de amostragem: 100 Hz (pasta `records100`, shape 1000×12 por registro) e 500 Hz (pasta `records500`, shape 5000×12 por registro). O arquivo `ptbx1_database.csv` contém todos os metadados, e o `scp_statements.csv` contém o dicionário de diagnósticos.

Decisão de frequência de amostragem: Use 100 Hz para a maior parte do pipeline de features clássicas — é suficiente para cobrir a banda diagnóstica do ECG (0,05–40 Hz, Nyquist exige mínimo 80 Hz) e reduz o custo computacional em $5\times$. Use 500 Hz apenas se for extrair features de micro-potenciais tardios, análise de alta frequência do QRS ou implementar CNNs que exploram resolução temporal fina. Esta decisão deve ser documentada e justificada no Entregável 1 e mantida consistente nos demais.

Sobre o Problema Multi-Label: O PTB-XL é um dataset multi-label. Cada registro pode ter múltiplos diagnósticos simultaneamente — por exemplo, um registro pode ser rotulado com MI (infarto) E CD (distúrbio de condução) ao mesmo tempo. A coluna `scp_codes` armazena um dicionário Python onde a chave é o código SCP e o valor é a verossimilhança (likelihood) de 0 a 100%. Para diagnósticos, a verossimilhança reflete a certeza do cardiologista (ex.: 50% = “provável”, 100% = “confirmado”). Para statements de forma e ritmo, a verossimilhança é sempre 0 (presença/ausência binária). Este comportamento

precisa ser explicitamente tratado ao definir labels para os algoritmos de RP.

Sobre os Folds de Validação Cruzada: O dataset fornece uma coluna `strat_fold` (inteiros de 1 a 10) que define a qual fold pertence cada registro. A lógica é: folds 1–8 são treino, fold 9 é validação, fold 10 é teste. Os folds 9 e 10 contêm exclusivamente registros com `validated_by_human = True` — label de maior qualidade. **Regra de ouro:** qualquer parâmetro aprendido a partir dos dados (media/std para normalização, componentes do PCA, limiares de filtros adaptativos) deve ser calculado exclusivamente nos registros dos folds 1–8 e apenas *aplicado* nos folds 9 e 10.

Convenção de Arquivos Salvos:

Notebook	Arquivo(s) de Saída
01	<code>ptbxtl_metadata_enriched.csv</code>
02	<code>ptbxtl_com_sqi.csv</code> , <code>rejected_ecg_ids.txt</code>
03	<code>estatistica_inicial_resultados.csv</code>
04	<code>sinais_limpos_100hz.npy</code> (ou HDF5), <code>preprocessing_params.pkl</code>
05	<code>janelas_segmentadas.npy</code> , <code>janelas_labels.npy</code> , <code>janelas_ids.csv</code>
06	<code>features_raw.parquet</code>
07	<code>features_engineered.parquet</code> , <code>scaler_params.pkl</code>
08	<code>features_pca.parquet</code> , <code>pca_model.pkl</code>
09	<code>features_final.parquet</code> , <code>feature_selection_report.csv</code>
10	<code>dataset_final_X.parquet</code> , <code>dataset_final_y.parquet</code> , <code>pipeline_params.pkl</code> , <code>folds_assignment.csv</code>

Padrão de Abertura — Aplicável a Todos os Notebooks

Todo notebook começa com as seguintes células, nesta ordem:

Célula 1 — Cabeçalho Markdown: Título do entregável, disciplina, instituição, nomes dos autores, data e número da versão.

Célula 2 — Objetivo: Parágrafo de 3–5 linhas descrevendo o propósito específico deste notebook dentro do pipeline, o que ele recebe como entrada e o que produz como saída.

Célula 3 — Importações e Dependências: Importação de todas as bibliotecas utilizadas no notebook, com comentário explicando o papel de cada uma. Bibliotecas esperadas ao longo do pipeline: `numpy`, `pandas`, `scipy`, `wfdb`, `matplotlib`, `seaborn`, `sklearn`, `pywt` (wavelets), `neurokit2` ou `biosppy` (ECG específico), `statsmodels`, `joblib` (serialização de modelos). Definir a semente aleatória global: `np.random.seed(42)`.

Célula 4 — Configuração Global: Definir como constantes: `PATH_DATA` (caminho raiz do PTB-XL), `FS` (frequência de amostragem escolhida: 100 ou 500), `N_LEADS` = 12, `LEAD_NAMES` = ['I', 'II', 'III', 'aVL', 'aVR', 'aVF', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6'], `FOLDS_TREINO` = [1, 2, 3, 4, 5, 6, 7, 8], `FOLD_VAL` = 9, `FOLD_TEST` = 10.

Célula 5 — Carregamento dos Metadados Base: Carregar `ptbxl_database.csv` com `index_col='ecg_id'`. Converter a coluna `scp_codes` de string para dicionário Python usando `ast.literal_eval`. Carregar `scp_statements.csv` com `index_col=0`. Estes dois DataFrames devem estar disponíveis em todas as células subsequentes.

ENTREGÁVEL 1 — Aquisição dos Biossinais e Caracterização do Dataset

Arquivo: 01_aquisicao_biossinais.ipynb

Objetivo: Documentar criticamente as condições de aquisição do dataset, caracterizar demograficamente e clinicamente a base de dados, e inspecionar visualmente o sinal bruto. Este notebook não produz dados transformados — apenas documentação, visualizações e o arquivo de metadados enriquecido.

Seção 1 — Contextualização do Dataset

Subseção 1.1 — O Sinal de ECG e sua Relevância Clínica

Escrever em markdown uma descrição técnica do ECG: é o registro da atividade elétrica do coração ao longo do tempo, medido por eletrodos posicionados na superfície corporal. As 12 derivações do ECG padrão fornecem perspectivas complementares do mesmo fenômeno elétrico. As derivações de membros (I, II, III, aVL, aVR, aVF) visualizam o plano frontal; as derivações precordiais (V1–V6) visualizam o plano horizontal. Cada derivação mostra a mesma sequência de eventos cardíacos — despolarização atrial (onda P), despolarização ventricular (complexo QRS) e repolarização ventricular (onda T) — a partir de um ângulo diferente.

Descrever por que 12 derivações são necessárias: certos diagnósticos são visíveis apenas em derivações específicas. Infarto anterior aparece em V1–V4; infarto

inferior aparece em II, III e aVF; bloqueios de ramo têm morfologia QRS característica em V1 e V6. Isso implica que o pipeline deve tratar as 12 derivações como dimensões complementares e não redundantes do mesmo sinal.

Subseção 1.2 — Protocolo de Aquisição Original

Organizar em tabela markdown os parâmetros técnicos do dataset: - **Equipamento:** dispositivos Schiller AG (fabricante suíço de equipamentos médicos) - **Período de coleta:** outubro de 1989 a junho de 1996 - **Instituição:** Physikalisch-Technische Bundesanstalt (PTB), Berlim, Alemanha - **Número de registros:** 21.837 registros de 10 segundos - **Número de pacientes:** 18.885 (alguns pacientes têm múltiplos registros — ver Seção 2.4) - **Derivações:** 12 derivações padrão (I, II, III, aVL, aVR, aVF, V1–V6), eletrodo de referência no braço direito - **Frequência original:** 400 Hz → reamostrado para 500 Hz por interpolação cúbica - **Versão downsampled:** 100 Hz (produzida por decimação do sinal a 500 Hz) - **Precisão do sinal:** 16 bits, resolução de 1 μ V/LSB (cada unidade de LSB corresponde a 1 microvolt) - **Duração por registro:** 10 segundos fixos (segmentos foram extraídos automaticamente dos registros contínuos originais) - **Formato de distribuição:** WFDB (WaveForm DataBase, padrão PhysioNet), dois arquivos por registro: `.dat` (dados binários brutos) e `.hea` (header com metadados de canal, ganho, baseline e unidades)

Subseção 1.3 — Justificativa do Critério de Nyquist

Explicar o Teorema de Nyquist-Shannon: para reconstruir perfeitamente um sinal com componentes de frequência até $f_{\max}$, a taxa de amostragem fs deve ser $fs \geq 2 \times f_{\max}$. Para ECG diagnóstico, a banda de interesse vai de 0,05 Hz (componente DC de baixa frequência, relevante para análise de ST) a 150 Hz (micro-potenciais tardios, relevantes em análises especializadas). Para análise diagnóstica clínica padrão (morfologia QRS, análise de ST-T, HRV), a banda de interesse é 0,05–40 Hz, exigindo $fs \geq 80$ Hz. Portanto, 100 Hz satisfaz o critério para análise diagnóstica padrão. Os 500 Hz são necessários para análise de alta frequência (QRS de alta frequência, micro-potenciais) ou para aplicações de deep learning que beneficiam de resolução temporal maior.

Documentar a decisão do projeto: qual frequência será usada e por quê.

Subseção 1.4 — Processo de Anotação dos Labels

Descrever em detalhe o processo de geração dos labels (esta informação é crítica para interpretar a confiabilidade dos rótulos):

O processo seguiu estas etapas para cada registro: 1. Um laudo inicial foi gerado por: (a) interpretação manual de um cardiologista humano — 67,13% dos registros; (b) interpretação automática pelo próprio dispositivo ECG, sem validação humana confirmada — 26,75% dos registros; (c) interpretação automática com validação por cardiologista humano — 4,45% dos registros; (d) sem laudo inicial — 1,67%. 2. O laudo em texto (escrito em 70,89% alemão, 27,9% inglês e 1,21% sueco) foi convertido para códigos SCP-ECG padronizados, com extração

de verossimilhança baseada em palavras-chave do laudo. 3. Um segundo cardiologista independente revisou e pôde corrigir os statements. 4. Um especialista técnico realizou anotação manual de qualidade do sinal (baseline drift, ruído, artefatos).

A coluna `validated_by_human` (booleano) resume o resultado: **True** indica que o registro passou por pelo menos uma validação humana confirmada (73,7% dos registros); **False** indica registro potencialmente anotado apenas automaticamente (26,3%).

Criar uma célula que calcula e exibe: `df['validated_by_human'].value_counts(normalize=True)` e `df['initial_autogenerated_report'].value_counts(normalize=True)`.

Subseção 1.5 — Verossimilhança dos Diagnósticos (Likelihood)

Explicar em detalhe como interpretar os valores em `scp_codes`. A coluna contém um dicionário como `{'NORM': 0, 'IMI': 100, 'IRBBB': 80}`. O valor é a verossimilhança do diagnóstico: - **0%**: statement de forma ou ritmo (presença binária, sem gradação de certeza), OR diagnóstico com certeza mínima - **15%**: expressão de dúvida (“não se pode excluir”, “cannot rule out”) - **35%**: expressão de possibilidade (“considerar”, “sugerir”, “provável”) - **50%**: expressão de probabilidade (“possivelmente”, “talvez”) - **80%**: expressão de quase certeza (“imagem compatível com”) - **100%**: diagnóstico confirmado (“consistente com”, “diagnóstico de”)

Para fins de classificação binária ou multi-classe, a decisão padrão recomendada é usar `threshold` de `likelihood` 50 para considerar um diagnóstico como positivo. Esta decisão deve ser documentada e aplicada de forma consistente ao longo de todo o pipeline. Criar uma função utilitária que será importada nos notebooks seguintes: recebe o dicionário `scp_codes` e o limiar, retorna uma lista de superclasses presentes.

Subseção 1.6 — Estrutura do Arquivo de Metadados

Exibir e comentar cada coluna do `ptb1_database.csv`:

Identificadores: - `ecg_id` — identificador único do registro ECG (inteiro, é o índice da tabela) - `patient_id` — identificador único do paciente (um paciente pode ter múltiplos `ecg_id`) - `filename_lr` — caminho relativo para o arquivo WFDB a 100 Hz (ex: `records100/00000/00001_lr`) - `filename_hr` — caminho relativo para o arquivo WFDB a 500 Hz (ex: `records500/00000/00001_hr`)

Metadados gerais: - `age` — idade em anos no momento do ECG (pacientes com idade 90 têm valor 300 por anonimização HIPAA) - `sex` — sexo (0 = masculino, 1 = feminino) - `height` — altura em centímetros (presente em 31,98% dos registros) - `weight` — peso em quilogramas (presente em 43,18% dos registros) - `nurse` — identificador pseudonimizado da enfermeira responsável pelo registro - `site` — identificador pseudonimizado do local de coleta (total de 51 sites distintos) - `device` — tipo de dispositivo utilizado (11 tipos distintos) - `recording_date` — data e hora do registro no formato YYYY-MM-DD

hh:mm:ss (deslocadas aleatoriamente por paciente para anonimização, preservando diferenças temporais entre registros do mesmo paciente)

Statements ECG: - **report** — laudo textual original escrito pelo cardiologista - **scp_codes** — dicionário {código_SCP: likelihood} com todos os diagnósticos, formas e ritmos identificados - **heart_axis** — eixo elétrico do coração no sistema de Cabrera (presente em 61,05% dos registros) - **infarction_stadium1** e **infarction_stadium2** — estágio do infarto quando presente (Stadium I = agudo/recente, II = subagudo, III = crônico/antigo) - **validated_by** — identificador pseudonimizado do cardiologista validador - **second_opinion** — booleano, True quando há uma segunda opinião registrada - **initial_autogenerated_report** — booleano, True quando o laudo inicial foi gerado automaticamente pelo dispositivo - **validated_by_human** — booleano, estimativa conservadora de validação humana

Metadados de qualidade do sinal: - **baseline_drift** — presença de drift ou salto de linha de base (string descritiva, NaN se ausente) - **static_noise** — presença de ruído estático / zumbido elétrico (string descritiva) - **burst_noise** — presença de picos de ruído burst (string descritiva) - **electrodes_problems** — problemas com eletrodos (string descritiva) - **extra_beats** — extrassístoles anotadas manualmente (presente em 8,95% dos registros) - **pacemaker** — padrão de marcapasso identificado no sinal (presente em 1,34% dos registros)

Folds: - **strat_fold** — número do fold de 1 a 10 para validação cruzada estratificada

Seção 2 — Panorama Estatístico do Dataset

Subseção 2.1 — Distribuição Demográfica

Calcular e apresentar em células separadas:

Para **idade**: histograma com bins de 5 anos, separado por sexo (cores distintas, transparência 0.7 para sobreposição). Linha vertical marcando a mediana (62 anos). Calcular explicitamente: `df['age'].replace(300, np.nan).describe()` — os 300 anos (anonimização de idosos 90) devem ser tratados como NaN para análises de distribuição. Caixa de texto no gráfico com mediana, IQR (22 anos), mínimo (0) e máximo real (95).

Para **sexo**: gráfico de pizza com 52% masculino e 48% feminino. Calcular: `df['sex'].value_counts(normalize=True)`.

Para **peso e altura**: histogramas com KDE sobreposta, indicando percentual de dados presentes (31,98% para altura, 43,18% para peso). Plotar scatter de peso $\times$ altura com coloração por sexo para verificar consistência.

Para **site e dispositivo**: gráficos de barra do número de registros por site (anonimizado) e por dispositivo. Verificar se há dominância de algum site que

poderia introduzir viés.

Subseção 2.2 — Criação das Colunas de Superclasse Diagnóstica

Esta é uma operação fundamental que será usada em todo o pipeline. Implementar com explicação detalhada:

Carregar `scp_statements.csv` e filtrar apenas as linhas onde `diagnostic == 1`. Criar um dicionário de mapeamento de código SCP para superclasse diagnóstica (ex: 'IMI': 'MI', 'NORM': 'NORM', 'LAFB': 'CD', etc.).

Escrever uma função `get_superclasses(scp_dict, threshold=50)` que: recebe o dicionário de `scp_codes` de um registro e o limiar de verossimilhança; retorna uma lista de superclasses cujos diagnósticos têm `likelihood > threshold`; se nenhum diagnóstico diagnóstico (não de forma/ritmo) atinge o `threshold`, retorna lista vazia.

Aplicar esta função ao DataFrame: `df['diagnostic_superclass'] = df['scp_codes'].apply(lambda x: get_superclasses(x, threshold=50))`.

Discutir explicitamente o problema multi-label: um registro pode pertencer a mais de uma superclasse simultaneamente. Calcular e exibir: quantos registros pertencem a exatamente 0, 1, 2, 3 ou mais superclasses. Registros com 0 superclasses diagnósticas são tipicamente registros de marcapasso ou com statements apenas de forma/ritmo.

Subseção 2.3 — Distribuição de Labels Diagnósticos

Contar a frequência de cada superclasse considerando o problema multi-label (um registro é contado em cada superclasse em que aparece). Calcular: para cada superclasse S, contar quantos registros têm S em sua lista de superclasses.

Apresentar: - Gráfico de barra horizontal com frequência absoluta por superclasse, separado por sexo (stacked bar ou grouped bar) - Tabela com: superclasse, n_registros, percentual do total, split masculino/feminino, mediana de idade - NORM: 9.528 registros; MI: ~5.486; CD: ~4.907; STTC: ~5.250; HYP: ~2.655 (verificar nos dados reais)

Para as subclasses diagnósticas, fazer o mesmo procedimento mas usando `scp_statements.csv` para o mapeamento código→subclasse. Criar um painel de 5 subplots (um por superclasse) com gráficos de barra das subclasses dentro de cada superclasse, separados por sexo.

Subseção 2.4 — Distribuição de Statements de Forma e Ritmo

Para **ritmo**: extrair do `scp_codes` apenas os códigos marcados como `rhythm == 1` no `scp_statements.csv`. Os principais são: SR (sinus rhythm) com 16.782 registros, AFIB (atrial fibrillation) com 1.514, STACH (sinus tachycardia) com 826, SARRH (sinus arrhythmia) com 772, SBRAD (sinus bradycardia) com 637, PACE (pacemaker) com 296. Plotar gráfico de barra e discutir: o ritmo sinusal normal (SR) domina. AFIB é a arritmia mais comum no dataset.

Para **forma**: extrair os códigos com `form == 1`. Apresentar tabela com frequências. ABQRS (QRS anormal) com 3.327 e PVC (complexo ventricular prematuro) com 1.146 são os mais frequentes.

Subseção 2.5 — Multiplicidade de Registros por Paciente

Calcular: `records_per_patient = df.groupby('patient_id')['ecg_id'].count().value_counts().sort_index()`
Apresentar tabela mostrando: 16.758 pacientes têm 1 registro, 1.604 têm 2 registros, 348 têm 3, e assim por diante (10 registros máximo para 1 paciente).

Discutir implicação crítica: a coluna `strat_fold` garante que todos os registros de um mesmo paciente estejam no mesmo fold. Isso evita data leakage por dependência temporal (um modelo não deve ver um ECG de treino de um paciente e depois ser avaliado em outro ECG do mesmo paciente no teste). Verificar isso programaticamente: `df.groupby('patient_id')['strat_fold'].nunique().max()` deve retornar 1.

Seção 3 — Carregamento e Visualização do Sinal Bruto

Subseção 3.1 — Função de Carregamento com wfdb

Implementar e explicar a função de carregamento:

```
load_ecg(ecg_id, df, path, fs=100):  
    filename = df.loc[ecg_id, 'filename_lr'] se fs=100, else 'filename_hr'  
    signal, fields = wfdb.rdsamp(path + filename)  
    return signal # shape: (1000, 12) para 100Hz ou (5000, 12) para 500Hz
```

Explicar os campos do objeto `fields` retornado pelo wfdb: `fs` (frequência de amostragem), `sig_name` (nomes das derivações), `units` (unidades — deve ser 'mV'), `n_sig` (número de sinais = 12), `sig_len` (comprimento = 1000 ou 5000). Verificar que as unidades são mV e que não há saturação (valores fora do range ± 5 mV são suspeitos para ECG de superfície).

Subseção 3.2 — Seleção de Registros Representativos

Selecionar especificamente os seguintes registros para visualização, justificando cada escolha: - Um registro da superclasse NORM com `validated_by_human = True` e todos os campos de qualidade NaN (exemplo de sinal ideal) - Um registro de MI (infarto do miocárdio) com alta likelihood (100%), mostrando supradesnívelamento de ST - Um registro de AFIB (fibrilação atrial), que na forma típica mostrará ausência de onda P e irregularidade dos intervalos RR - Um registro com `baseline_drift` não nulo (exemplo de artefato de linha de base) - Um registro com `static_noise` não nulo (exemplo de ruído elétrico) - Um registro de PACE (marcapasso ativo), reconhecível pelos spikes de estimulação

Para cada um, exibir: `ecg_id`, `patient_id`, `age`, `sex`, `scp_codes`, `validated_by_human` e os campos de qualidade.

Subseção 3.3 — Visualização das 12 Derivações

Para cada registro selecionado, plotar as 12 derivações organizadas no layout clínico padrão: duas linhas, primeira linha com I, II, III, aVL, aVR, aVF (derivações de membros), segunda linha com V1, V2, V3, V4, V5, V6 (derivações precordiais). Cada subplot deve ter: eixo x em segundos (converter amostras: `t = np.arange(len(signal)) / fs`), eixo y em mV, grade leve, título com nome da derivação, e linha preta de espessura 0.8. O título geral da figura deve conter o `ecg_id` e o diagnóstico principal. Definir os limites do eixo y como fixos (-2 mV a +2 mV) para permitir comparação visual entre registros.

Subseção 3.4 — Análise Espectral Visual do Sinal Bruto

Para os mesmos registros selecionados, calcular e plotar o espectro de potência da derivação DII usando o método de Welch (`scipy.signal.welch` com janela Hann de 256 amostras, overlap 50%): eixo x de 0 a 50 Hz (frequência), eixo y em dB ($10 \cdot \log_{10}$ da PSD). Marcar visualmente as frequências-chave: 0,5 Hz (limite inferior do bandpass pretendido), 40 Hz (limite superior), e 50 Hz (frequência da rede elétrica europeia). Isso servirá como “antes” para comparar com o espectro após filtragem no Entregável 4.

Seção 4 — Enriquecimento do Metadado e Salvamento

Subseção 4.1 — Adição de Colunas Calculadas

Adicionar ao DataFrame as seguintes colunas calculadas neste notebook: - `diagnostic_superclass` — lista de superclasses com likelihood 50 (criada na Seção 2.2) - `n_superclasses` — número de superclasses simultâneas por registro - `is_pure_norm` — booleano: True somente para registros com `diagnostic_superclass == ['NORM']` (apenas NORM, sem outras patologias) - `has_quality_issues` — booleano: True se qualquer campo de qualidade (`baseline_drift`, `static_noise`, `burst_noise`, `electrodes_problems`) for não nulo - `split` — string: ‘train’ para `strat_fold` in [1–8], ‘val’ para `strat_fold == 9`, ‘test’ para `strat_fold == 10`

Subseção 4.2 — Salvamento

Salvar o DataFrame enriquecido como `ptbxl_metadata_enriched.csv`. Imprimir um resumo final: número de registros por split (treino/val/teste), número de registros com qualidade issues, distribuição de `n_superclasses`.

Seção 5 — Síntese e Conexão

Texto-resumo confirmando o que foi estabelecido: o dataset está carregado e inspecionado, os metadados foram enriquecidos, o protocolo de aquisição foi documentado, o sinal bruto foi visualizado com identificação de padrões normais

e patológicos. A decisão sobre frequência de amostragem está registrada. O próximo passo (Entregável 2) quantificará objetivamente a qualidade dos sinais usando métricas computadas diretamente sobre os arrays de sinal.

ENTREGÁVEL 2 — Avaliação da Qualidade do Sinal (SQI)

Arquivo: 02_qualidade_sinal_SQI.ipynb

Objetivo: Atribuir a cada registro um índice objetivo de qualidade de sinal (SQI), combinando múltiplas métricas computadas sobre o sinal bruto. Decidir quais registros serão rejeitados ou marcados para tratamento especial antes de qualquer processamento.

Entrada: `ptb_xl_metadata_enriched.csv`, sinais brutos em `records100/` (ou `records500/`)

Seção 1 — Estratégia de Amostragem para Cálculo do SQI

Subseção 1.1 — Por que não calcular SQI para todos os 21.837 registros de uma vez

Calcular SNR, entropia espectral e kurtosis para todos os registros é computacionalmente viável, mas deve ser feito de forma eficiente. Implementar processamento em lote (batch): dividir os registros em grupos de 500, processar cada grupo, salvar resultados parciais. Usar `tqdm` para barra de progresso. Estimar o tempo de processamento para um registro e multiplicar pelo total.

Subseção 1.2 — Verificação de Consistência dos Metadados de Qualidade

Antes de calcular as métricas, analisar os campos de qualidade existentes no metadado: - Calcular: `df['has_quality_issues'].value_counts()` — quantos registros têm ao menos um problema anotado - Calcular: percentual de registros sem nenhum problema (deve ser ~77,01%) - Criar tabela cruzada entre cada campo de qualidade e a superclasse diagnóstica: `pd.crosstab(df['diagnostic_superclass'].apply(lambda x: x[0] if len(x)==1 else 'multi'), df['has_quality_issues'])` — verificar se problemas de qualidade se concentram em alguma classe - Criar tabela cruzada entre qualidade e `strat_fold` — verificar se problemas se concentram em períodos temporais específicos (o dataset foi coletado em 6+ anos; dispositivos e protocolos mudaram)

Seção 2 — Implementação das Métricas de SQI

Subseção 2.1 — SNR (Signal-to-Noise Ratio)

O SNR será calculado por derivação e por registro. O método adotado: 1. Aplicar filtro passa-banda Butterworth de 4ª ordem (0,5–40 Hz) ao sinal bruto para obter uma estimativa do sinal limpo `s_clean` 2. Calcular o ruído como `noise = signal_raw - s_clean` 3. Calcular $\text{SNR}_{\text{dB}} = 10 * \log_{10}(\text{RMS}(\text{s_clean})^2 / \text{RMS}(\text{noise})^2)$ Onde $\text{RMS}(x) = \sqrt{\text{mean}(x^2)}$

Calcular o SNR para cada uma das 12 derivações de cada registro. Armazenar como array 21.837×12 . Para o índice por registro, usar a mediana do SNR entre as 12 derivações (a mediana é preferível à média por ser robusta a uma derivação muito ruidosa).

Apresentar: - Histograma da distribuição de SNR mediano por registro (em dB) - Boxplot do SNR por derivação (verificar se alguma derivação é sistematicamente mais ruidosa) - Scatter plot de SNR vs. `strat_fold` para verificar tendências temporais - Definir limiar: SNR ≥ 5 dB $\rightarrow$ alta/boa qualidade. Justificar: O limite de 5 dB em sinais de 100 Hz compensa o corte do bandpass em 40 Hz, que acaba isolando o conteúdo de alta frequência do próprio QRS (40–150 Hz) no resíduo de “ruído”. Valores < 5 dB indicam que o ruído verdadeiro de fato excede o sinal base.

Subseção 2.2 — Kurtosis

Kurtosis mede o “peso das caudas” da distribuição de amplitude do sinal. Para um sinal ECG limpo e normal, a kurtosis é moderada (próxima de 3 para distribuição gaussiana). Kurtosis muito alta (> 10) indica presença de spikes abruptos (artefatos de movimento, eletrodo momentaneamente desconectado). Kurtosis muito baixa (< 1.5) pode indicar saturação de sinal.

Calcular usando `scipy.stats.kurtosis(signal, axis=0, fisher=True)` — kurtosis de Fisher (excesso, onde gaussiana = 0). Calcular por derivação e usar a mediana entre derivações por registro. Plotar histograma da kurtosis por registro e boxplot por derivação. Limiares: kurtosis entre 1 e 50 $\rightarrow$ sinal com picos QRS fisiológicos saudáveis (leptocúrtico). Valores < 1 (muito planos) ou > 50 (spikes de RF extremos) $\rightarrow$ flag de falha estrutural.

Subseção 2.3 — Skewness

Assimetria da distribuição de amplitude. ECG saudável possui naturalmente alta assimetria (ex: DII foca no eixo principal do coração gerando onda R alta e positiva com onda S menor). Calcular `scipy.stats.skew(signal, axis=0)` por derivação. Usar mediana entre derivações. Limiar: $|\text{skewness}| > 5.0 \rightarrow$ flag indicando artefatos extremamente severos de baixa frequência ou offset anormal.

Subseção 2.4 — Entropia Espectral

Mede a distribuição de energia no espectro de frequências. Um sinal limpo com componentes bem definidos (onda P, QRS, T) tem espectro concentrado

→ baixa entropia. Um sinal dominado por ruído branco tem espectro plano → alta entropia.

Cálculo: 1. Calcular PSD via Welch: `freqs, psd = scipy.signal.welch(signal, fs=100, nperseg=256)` 2. Normalizar: `psd_norm = psd / psd.sum()` 3. Calcular entropia de Shannon: `H = -sum(psd_norm * log2(psd_norm + eps))` onde `eps=1e-10` evita `log(0)` 4. Normalizar pelo máximo teórico: `H_norm = H / log2(len(psd))` — resultado entre 0 (espectro impulsivo puro) e 1 (ruído branco puro)

Calcular por derivação DII (mais rica em componentes cardíacos). Limiar: `H_norm > 0.95` → flag de ruído dominante. (Nota: limiares muito restritivos como 0.85 penalizariam arritmias reais como a fibrilação atrial - AFIB - que naturalmente espalham o espectro diluindo picos harmônicos e elevando a entropia fisiológica para ~0.87-0.93).

Subseção 2.5 — Detecção de Saturação

Um sinal saturado apresenta longos períodos de amplitude constante (o ADC do dispositivo atingiu seu limite). Detectar: calcular a proporção de amostras consecutivas com diferença zero, ou seja, `flat_ratio = mean(diff(signal) == 0)`. Se `flat_ratio > 0.05` (mais de 5% das amostras consecutivas idênticas), o sinal é suspeito de saturação. Calcular por derivação, flag se qualquer derivação exceder o limiar.

Subseção 2.6 — Detecção de Ruído de Rede Residual (50 Hz)

Calcular a PSD via Welch. Extrair a potência na banda de 49–51 Hz e normalizar pela potência total na banda de 10–40 Hz. Contudo, em sinais submetidos a `fs=100` Hz, a frequência de Nyquist coincide com os exatos 50 Hz, tornando a equação matematicamente degenerada (limitada por aliasing). A versão original de 500 Hz do PTB-XL aplicou um filtro antialiasing que mitigou os 50 Hz naturais na etapa de decimação. Logo, este valor é calculado por completude de pipeline em 100 Hz, não possuindo utilidade limitadora para compor o score combinado ou forçar rejeições sistêmicas.

Seção 3 — SQI Composto e Critério de Decisão

Subseção 3.1 — Cálculo do SQI Composto

Combinar as 5 métricas em um score único. Estratégia: atribuir pontuação binária (0 ou 1) a cada critério e somar: - SNR mediano ≥ 5 dB → +1 - Kurtosis de Fisher entre 1 e 50 → +1 - $|\text{Skewness}| \leq 5.0$ → +1 - `H_norm` ≥ 0.95 → +1 - `flat_ratio` < 0.05 → +1

`sqi_score = (soma dos critérios) / 5` → varia de 0.0 a 1.0.

Justificar esta abordagem: cada critério detecta um tipo diferente de problema, e a soma não ponderada trata todos como igualmente importantes. Discutir

alternativa: dar peso 2 ao SNR (mais informativo) e peso 1 aos demais — calcular ambas as versões e comparar o ranking resultante.

Subseção 3.2 — Categorias SQI e Critérios de Decisão

Definir três categorias: - **Categoria A (Alta qualidade):** `sqi_score >= 0.8` — todos ou quase todos os critérios satisfeitos → incluso normalmente no pipeline
- **Categoria B (Qualidade marginal):** `0.6 <= sqi_score < 0.8` — alguns critérios falhos → incluso com flag, processado com cuidado extra
- **Categoria C (Baixa qualidade):** `sqi_score < 0.6` — múltiplos critérios falhos → rejeitado do pipeline de features; pode ser incluído apenas em análises de robustez

Adicionar ao DataFrame: `sqi_score` (float), `sqi_category` (string 'A', 'B', 'C'). Contar e exibir: quantos registros em cada categoria, separados por split treino/val/teste.

Importante: Registros do fold 10 (teste) que sejam categoria C devem ser anotados mas não necessariamente removidos da análise — a qualidade do conjunto de teste afeta a avaliação dos modelos de RP e deve ser reportada.

Subseção 3.3 — Validação Cruzada com Metadados do PTB-XL

Calcular a concordância entre o SQI calculado e os campos de qualidade originais: - Dos registros com `has_quality_issues == True`, qual proporção é categoria C ou B? - Dos registros com `has_quality_issues == False`, qual proporção é categoria A? - Calcular a taxa de falsos positivos (SQI = C mas metadado = sem problemas) e falsos negativos (SQI = A mas metadado = com problemas)

Isso mostra que o SQI calculado é consistente com a avaliação humana original, validando a metodologia.

Seção 4 — Visualizações Comparativas

Subseção 4.1 — Painel Comparativo Categoria A vs. C

Montar figura com 6 linhas e 3 colunas: linhas representam 3 registros de categoria A e 3 de categoria C; colunas mostram respectivamente: (1) sinal bruto da derivação DII, (2) espectro de potência com marcações em 0.5 Hz, 40 Hz e 50 Hz, (3) histograma da distribuição de amplitude. Anotar em cada linha os valores de SNR, kurtosis, H_norm e `sqi_score`. Esta figura é a evidência visual principal do entregável.

Subseção 4.2 — Mapa de Calor de Métricas SQI

Criar heatmap de correlação entre as 5 métricas de SQI: SNR, kurtosis, skewness, entropia espectral, `flat_ratio`. Métricas altamente correlacionadas detectam o mesmo problema e um deles poderia ser removido. Discutir quais métricas são independentes (SNR vs. entropia espectral geralmente capturam coisas distintas).

Subseção 4.3 — Distribuição do SQI por Derivação

Violin plot ou boxplot do SNR separado por derivação (12 derivações no eixo x, SNR em dB no eixo y). Identificar derivações sistematicamente mais ruidosas — frequentemente V1 e V2 (precordiais anteriores) têm mais artefatos de movimento.

Seção 5 — Salvamento e Síntese

Salvar `ptbxxl_com_sqi.csv` (DataFrame enriquecido com `sqi_score` e `sqi_category`). Salvar `rejected_ecg_ids.txt` com os `ecg_ids` de categoria C. Imprimir resumo: `n_A`, `n_B`, `n_C` por split, concordância com metadados originais em percentual.

A partir do Entregável 3, trabalhar apenas com registros de categoria A e B. Registrar explicitamente esta decisão.

ENTREGÁVEL 3 — Análise Estatística Inicial da Base

Arquivo: `03_estatistica_inicial.ipynb`

Objetivo: Caracterizar estatisticamente o comportamento bruto do dataset — metadados e sinais — antes de qualquer transformação. Os resultados orientam as escolhas metodológicas de todos os entregáveis seguintes: qual teste usar (paramétrico vs. não paramétrico), se é necessário transformar variáveis, quais correlações existem.

Entrada: `ptbxxl_com_sqi.csv`, sinais brutos

Seção 1 — Estatística Descritiva dos Metadados

Subseção 1.1 — Variáveis Demográficas (Dataset Completo)

Para `age` (com `300` → `NaN`), `weight` e `height`, calcular em uma única tabela: `n` (não nulos), média, mediana, desvio padrão, variância, mínimo, máximo, percentis 5, 25, 75, 95 e IQR. Usar `df[['age', 'weight', 'height']].describe(percentiles=[0.05, 0.25, 0.75])` e complementar com IQR e variância manualmente.

Plotar: histograma com KDE para cada variável, separado por sexo, com linhas verticais marcando mediana e ± 1 IQR. Boxplot horizontal por variável, com identificação de outliers (pontos além de $1.5 \times \text{IQR}$ como círculos individuais).

Subseção 1.2 — Variáveis Demográficas por Superclasse Diagnóstica

Para registros com exatamente 1 superclasse diagnóstica (simplificar a análise evitando ambiguidade multi-label), calcular a tabela descritiva de `age`, `weight`

e height segmentada por superclasse. Apresentar como tabela pivô: linhas = superclasses, colunas = estatísticas de cada variável. Identificar diferenças: por exemplo, MI tem maior mediana de idade do que NORM? HYP tem maior mediana de peso?

Plotar: boxplot de `age` com os grupos NORM, MI, CD, HYP, STTC no eixo x e a idade no eixo y. Verificar visualmente se as medianas diferem. Este será o pré-requisito para os testes de hipótese na Seção 4.

Subseção 1.3 — Distribuição Temporal dos Registros

Criar coluna `recording_year` a partir de `recording_date`. Plotar histograma de registros por ano. Sobrepor a fração de `initial_autogenerated_report` por ano — mostrar que o uso de laudos automáticos aumentou ao longo do período de coleta. Isso é relevante para entender a heterogeneidade temporal do dataset.

Seção 2 — Estatística Descritiva dos Sinais

Subseção 2.1 — Estatísticas de Amplitude por Derivação

Para uma amostra estratificada de 500 registros (100 por superclasse — selecionar aleatoriamente apenas dos folds 1–8, categoria SQI A), calcular por derivação e por registro: amplitude mínima, máxima, média, RMS e amplitude pico-a-pico. Calcular a mediana dessas métricas dentro de cada superclasse diagnóstica.

Apresentar como heatmap: eixo y = 12 derivações, eixo x = superclasses diagnósticas, células = amplitude RMS mediana. Isso revela quais derivações são mais energéticas para cada patologia — por exemplo, MI inferior tem maior amplitude em II, III, aVF.

Subseção 2.2 — Energia Total do Sinal

Calcular energia total de cada registro como `E = sum(signal**2) / len(signal)` (energia média por amostra, equivalente ao quadrado do RMS). Plotar distribuição da energia por superclasse diagnóstica (violinplot ou boxplot). Identificar outliers de energia extremos que possam indicar artefatos não capturados pelo SQI.

Seção 3 — Testes de Normalidade

Subseção 3.1 — Shapiro-Wilk

Aplicar Shapiro-Wilk (`scipy.stats.shapiro`) nas variáveis: `age`, `weight`, `height`, e nas estatísticas de sinal (amplitude RMS por derivação, energia total). Limitação importante: Shapiro-Wilk funciona melhor para $n <$

5.000; para amostras maiores, usar subamostras de 500 registros selecionados aleatoriamente.

Organizar resultados em tabela: variável, n, estatística W, p-valor, interpretação ($p < 0.05 \rightarrow$ rejeita normalidade). Calcular para o dataset total e também separado por superclasse.

Subseção 3.2 — Kolmogorov-Smirnov

Aplicar K-S test (`scipy.stats.kstest`) comparando cada variável com a distribuição normal ajustada à média e desvio padrão dos dados. O K-S é mais apropriado para amostras grandes. Apresentar na mesma tabela do Shapiro-Wilk para comparação direta. Discutir discordâncias: K-S rejeita normalidade mesmo quando Shapiro-Wilk aceita em grandes amostras, porque com n grande qualquer pequeno desvio da normalidade se torna estatisticamente significativo.

Subseção 3.3 — Q-Q Plots

Gerar Q-Q plots (`scipy.stats.probplot`) para: age, weight, height, RMS da derivação DII, energia total. Cinco subplots em uma figura. Para distribuições claramente não normais, tentar transformação log e plotar o Q-Q do log — se melhorar significativamente, registrar como possível transformação para o Entregável 7. Não aplicar ainda — apenas documentar.

Seção 4 — Testes de Homocedasticidade

Subseção 4.1 — Teste de Levene

Aplicar `scipy.stats.levene` para testar igualdade de variâncias entre os grupos de superclasse diagnóstica (usando registros com exatamente 1 superclasse) para cada variável de interesse. Levene usa desvios em relação à mediana de cada grupo, sendo robusto a não-normalidade. Tabela: variável, estatística de Levene, p-valor, interpretação.

Subseção 4.2 — Teste de Bartlett

Aplicar `scipy.stats.bartlett` nas mesmas variáveis. Bartlett assume normalidade e é mais poderoso quando essa condição é satisfeita. Comparar com Levene: se Levene rejeita homocedasticidade mas Bartlett não, provavelmente a não-normalidade está influenciando Bartlett. Se ambos rejeitam, a heterocedasticidade é robusta.

Subseção 4.3 — Decisão Metodológica Documentada

Com base nos Testes de Normalidade e Homocedasticidade, criar uma tabela-decisão:

Variável	Normal?	Homoscedástica	Teste para comparar grupos	Teste para 2 grupos
age	Não	Não	Kruskal- Wallis	Mann- Whitney U
RMS DII	A verificar	A verificar	ANOVA ou Kruskal- Wallis	t-test ou Mann- Whitney
...				

Esta tabela será referenciada nos Entregáveis 4 e 9.

Seção 5 — Análise de Correlação

Subseção 5.1 — Preparação da Matriz de Correlação

Criar um DataFrame com as variáveis candidatas à correlação: age, sex, weight, height, n_superclasses, sqi_score, RMS por derivação (12 colunas), energia total, e campos binários de qualidade. Total de ~20–25 variáveis.

Subseção 5.2 — Correlação de Pearson

Calcular a matriz de correlação de Pearson (`df.corr(method='pearson')`). Apresentar como heatmap `seaborn.heatmap` com: máscara triangular superior (evitar duplicação), anotação dos coeficientes (fonte pequena), colormap divergente (azul-branco-vermelho), limites de -1 a 1. Destacar com asterisco pares com $|r| > 0.7$ — candidatos à redundância. Destacar com bordas fortes pares com significância estatística ($p < 0.05$ após correção de Bonferroni).

Subseção 5.3 — Correlação de Spearman

Repetir com `method='spearman'`. Comparar lado a lado os dois heatmaps. Pares onde Pearson é baixo mas Spearman é alto indicam relações monotônicas não lineares — por exemplo, a relação entre idade e energia do sinal pode ser não linear (jovens e idosos têm padrões específicos).

Subseção 5.4 — Interpretação Fisiológica das Correlações

Escrever um parágrafo comentando as correlações mais fortes encontradas, buscando explicação clínica: - Correlação positiva entre age e probabilidade de MI é esperada (aterosclerose progressiva) - Correlação entre weight e HYP é esperada (obesidade como fator de risco para hipertrofia ventricular esquerda) - Correlação entre RMS de V1 e V2 (derivações adjacentes) é esperada por proximidade anatômica - Alta correlação entre RMS de várias derivações pode indicar que apenas alguns leads carregam informação única

Seção 6 — Síntese e Conexão

Criar uma tabela-síntese com as conclusões principais: quais variáveis são normais, quais não são, quais têm variâncias homogêneas entre classes, quais correlações foram encontradas. Salvar `estatistica_inicial_resultados.csv` com os resultados dos testes. Anotar explicitamente: “os testes revelaram que a maioria das variáveis não é normalmente distribuída → os testes de comparação nos Entregáveis 4 e 9 serão não paramétricos (Wilcoxon, Mann-Whitney, Kruskal-Wallis).”

ENTREGÁVEL 4 — Limpeza e Correção dos Dados

Arquivo: `04_limpeza_dados.ipynb`

Objetivo: Remover ruídos instrumentais e artefatos dos sinais sem destruir informação fisiológica. Cada técnica aplicada deve ter justificativa fisiológica e ser validada estatisticamente por comparação antes/depois.

Entrada: `ptbx1_com_sqi.csv`, sinais brutos; usar apenas registros de categoria SQI A e B

Seção 1 — Inventário de Problemas e Plano de Ação

Subseção 1.1 — Quadro de Problemas e Técnicas

Criar uma tabela antes de qualquer código:

Problema Identificado	Fonte de Evidência	Técnica de Correção	Parâmetros
Interferência de rede 50 Hz	Entregável 2 (verificação espectral)	Não aplicável a 100 Hz (ver Subseção 2.1)	—
Drift de linha de base	Campos <code>baseline_drift</code> no metadado	Filtro passa-alta	fc=0.5 Hz, Butterworth 4 ^a ordem
Ruído de alta frequência > 40 Hz	SNR baixo em banda > 40 Hz	Filtro passa-baixa	fc=40 Hz, Butterworth 4 ^a ordem
Spikes de liga/desliga	Detectados no início/fim dos registros	Remoção direta das primeiras/últimas 50 amostras (0.5s a 100Hz)	

Problema Identificado	Fonte de Evidência	Técnica de Correção	Parâmetros
Outliers de amplitude extremos	Kurtosis alta (Entregável 2)	Winsorização a 99.5%	
Gaps curtos de sinal	flat_ratio por segmento	Interpolação cúbica se $< 0.5s$	

Subseção 1.2 — Seleção da Amostra de Referência para Validação

Para validar o efeito dos filtros, criar três grupos de registros de referência: - **Grupo “Limpo”**: 100 registros com SQI categoria A, `has_quality_issues == False`, de superclasse NORM, dos folds 1–4. Estes devem ter mínima alteração após filtragem. - **Grupo “Ruído 50Hz”**: todos os registros com sinal de interferência de rede detectado no Entregável 2 (flag de 50 Hz positivo). Estes serão usados para confirmar, por verificação espectral, que a potência a 50 Hz já está ausente no sinal a 100 Hz — evidenciando que o anti-aliasing da decimação cumpriu seu papel. Não haverá filtro aplicado a este grupo; a análise é de sanidade, não de correção. - **Grupo “Baseline”**: todos os registros com `baseline_drift` não nulo. Estes devem mostrar melhoria após passa-alta.

Seção 2 — Filtragem do Sinal

Subseção 2.1 — Interferência de 50 Hz: por que o filtro notch não se aplica a 100 Hz

Justificativa fisiológica e instrumental: a interferência eletromagnética da rede elétrica europeia entra nos eletrodos a exatamente 50 Hz e não carrega informação cardíaca. Em princípio, deveria ser removida por um filtro notch IIR centrado em 50 Hz.

Por que não aplicar o notch na versão de 100 Hz: a frequência de Nyquist de um sinal amostrado a 100 Hz é exatamente 50 Hz. Um filtro notch a 50 Hz com `fs=100` operaria no limite absoluto do espectro representável — condição degenerada para o design de filtros IIR, onde a frequência normalizada $w0/(fs/2) = 50/50 = 1.0$ invalida a formulação matemática do filtro. Mais importante ainda: conforme descrito no artigo do PTB-XL (Wagner et al., 2020), a versão de 100 Hz foi gerada por decimação do sinal original de 500 Hz. Todo processo de decimação requer a aplicação prévia de um filtro anti-aliasing com corte bem abaixo de 50 Hz (tipicamente 40–45 Hz), o que já elimina ou atenua fortemente qualquer componente a 50 Hz antes que o sinal chegue ao usuário. Portanto, a interferência de rede elétrica já foi tratada no processo de geração do dataset — aplicar um novo notch seria tecnicamente inválido e desnecessário.

Decisão documentada: neste pipeline, operando a 100 Hz, o passo de filtro notch é suprimido. Documentar explicitamente esta decisão no relatório final (Entregável Final Integrador), citando o raciocínio acima.

Verificação de sanidade obrigatória: mesmo sem aplicar o filtro, verificar empiricamente a ausência de pico a 50 Hz. Calcular a PSD via Welch para 20 registros do Grupo “Ruído 50Hz” (anotados com `static_noise` não nulo) e confirmar que a potência na banda 48–50 Hz não ultrapassa em mais de 3 dB a potência nas bandas adjacentes (45–48 Hz e 40–44 Hz). Se houver registros com pico proeminente a 50 Hz mesmo após a decimação (o que seria anômalo), documentar como limitação do dataset, não como falha do pipeline.

Nota para projetos futuros com sinal bruto a 500 Hz: nesse caso, o filtro notch é aplicável e necessário. Implementar com `scipy.signal.iirnotch(w0=50, Q=30, fs=500)` seguido de `scipy.signal.filtfilt(b, a, signal, axis=0)` — processamento bidirecional que elimina defasagem de fase, preservando a morfologia das ondas cardíacas.

Subseção 2.2 — Filtro Passa-Alta para Remoção de Drift de Baseline

Justificativa fisiológica: variações lentas de amplitude (abaixo de 0,5 Hz) são causadas por respiração, movimentos do paciente e polarização dos eletrodos — não são atividade cardíaca. O segmento ST do ECG (cl clinicamente importante para diagnóstico de infarto) está acima de 0,5 Hz, portanto um corte em 0,5 Hz não o compromete. Não usar corte muito agressivo (ex: 1 Hz) pois poderia distorcer o segmento ST e a onda T.

Implementação: 1. Projetar filtro Butterworth de 4ª ordem com corte a 0,5 Hz: `scipy.signal.butter(N=4, Wn=0.5/(fs/2), btype='highpass')` 2. Aplicar com `filtfilt` em todas as derivações

Verificação visual: plotar sinal antes e depois para 5 registros do Grupo “Baseline”. Confirmar que a linha de base está mais estável (centrada em 0 mV) sem distorção do complexo QRS.

Subseção 2.3 — Filtro Passa-Baixa para Remoção de Ruído de Alta Frequência

Justificativa fisiológica: componentes acima de 40 Hz no ECG de superfície incluem ruído muscular (EMG), interferência elétrica de alta frequência e ruído do amplificador. O complexo QRS tem energia predominante até ~30–35 Hz; componentes acima de 40 Hz raramente são diagnósticos (exceto micro-potenciais tardios em análises especializadas, que não é o caso aqui).

Implementação: 1. Projetar Butterworth de 4ª ordem com corte a 40 Hz: `scipy.signal.butter(N=4, Wn=40/(fs/2), btype='lowpass')` 2. Aplicar com `filtfilt`

Nota técnica: se a frequência de amostragem escolhida for 100 Hz, o corte a 40 Hz está em 0.8 da frequência de Nyquist (50 Hz), o que é seguro. Se usar 500

Hz, o corte a 40 Hz está em 0.16 da Nyquist, igualmente seguro.

Subseção 2.4 — Pipeline de Filtragem Integrada

Combinar os dois filtros em uma única função `preprocess_signal(signal, fs)` que aplica sequencialmente: passa-alta 0,5 Hz → passa-baixa 40 Hz. O filtro notch de 50 Hz não é aplicado na versão de 100 Hz por razões documentadas na Subseção 2.1. Plotar o espectro médio do dataset antes e depois da pipeline completa (média dos espectros de 100 registros) para mostrar o efeito global: platô plano de 0,5–40 Hz, atenuação forte fora dessa banda.

Seção 3 — Tratamento de Outliers de Amplitude

Subseção 3.1 — Detecção por Z-score e MAD

Para cada registro filtrado, calcular por derivação: - Z-score de cada amostra: $z = (x - \text{mean}(x)) / \text{std}(x)$ - MAD normalizado: $\text{mad_score} = |x - \text{median}(x)| / (1.4826 * \text{MAD}(x))$ — o fator 1.4826 torna o MAD equivalente ao desvio padrão para distribuição normal

O Z-score é sensível a outliers por ser calculado com média e std. O MAD é robusto porque usa mediana. Usar $\text{MAD} > 3.5$ como critério (equivalente a aproximadamente ± 3.5 desvios padrão).

Calcular a proporção de amostras outliers por registro e por derivação. Criar histograma desta proporção — espera-se que a maioria dos registros tenha menos de 0.5% de outliers.

Subseção 3.2 — Winsorização

Para amostras identificadas como outliers pelo MAD, aplicar clip nos percentis 0,5% e 99,5% calculados do **conjunto de treino** (folds 1–8). Os limiares calculados no treino devem ser salvos em `preprocessing_params.pkl` e aplicados também no val/teste.

Importante: não usar percentis calculados por registro individual — isso suavizaria assimetrias fisiológicas reais. Calcular percentis por derivação a partir do conjunto de treino completo.

Subseção 3.3 — Remoção dos Spikes de Liga/Desliga

O dataset original já menciona que spikes de início e fim de gravação foram tratados, mas verificar empiricamente: calcular a variância das primeiras e últimas 50 amostras (0,5 s) de cada registro comparada com a variância das 100 amostras centrais. Se a razão $\text{var_extremos} / \text{var_centro} > 3$, remover os primeiros e últimos 50 pontos e preencher com interpolação linear.

Seção 4 — Validação Estatística: Antes vs. Depois

Subseção 4.1 — Métricas de Comparação

Para cada registro nos três grupos de referência (Limpo, Ruído 50Hz, Baseline), calcular antes e depois da pipeline completa: - RMS por derivação - Energia total - SNR (recalculando com os próprios dados filtrados como referência — usar filtro mais agressivo 1–30 Hz como proxy de “sinal limpo puro”) - Amplitude pico-a-pico por derivação

Criar arrays `before[n_registros, n_métricas]` e `after[n_registros, n_métricas]`.

Subseção 4.2 — Testes Estatísticos Pareados

Como os dados tipicamente não são normais (confirmado no Entregável 3), usar: - **Wilcoxon signed-rank test** (`scipy.stats.wilcoxon`) para comparar before vs. after de cada métrica dentro de cada grupo - Apresentar tabela: métrica, grupo, estatística W, p-valor, interpretação

Meta: para o Grupo “Limpo”, nenhuma métrica deve ter mudança significativa ($p > 0.05$) — confirma que o filtro não destrói sinal saudável. Para o Grupo “Ruído 50Hz”, a métrica de potência em 49–51 Hz deve mudar significativamente ($p < 0.001$). Para o Grupo “Baseline”, o RMS e a amplitude pico-a-pico devem mudar significativamente.

Subseção 4.3 — Effect Size (Cohen’s d)

Calcular $d = (\text{mean_before} - \text{mean_after}) / \text{pooled_std}$ para cada métrica e grupo.

Interpretar: $d < 0.2$ = trivial (filtro não alterou quase nada — desejável para Grupo Limpo); $d 0.2\text{--}0.5$ = pequeno; $d 0.5\text{--}0.8$ = médio; $d > 0.8$ = grande (filtro corrigiu algo substancial — desejável para Grupos Ruído e Baseline).

Criar tabela comparativa side-by-side: Grupo Limpo (esperado: d trivial em todas métricas), Grupo Ruído 50Hz (esperado: d grande em potência 50 Hz, trivial nas demais), Grupo Baseline (esperado: d grande no RMS em baixa frequência, trivial nas demais). Esta assimetria demonstra seletividade da pipeline de limpeza.

Seção 5 — Processamento de Todo o Dataset e Salvamento

Subseção 5.1 — Aplicação em Batch

Processar todos os registros de categoria A e B em batches de 500, aplicando a pipeline completa: passa-alta → passa-baixa → winsorização. Usar `tqdm` para acompanhar progresso. Para cada batch, verificar que não há NaN ou Inf no output.

Subseção 5.2 — Formato de Armazenamento

Salvar o dataset de sinais limpos no formato HDF5 (`h5py`) ou numpy comprimido (`.npz`), organizado por `ecg_id`. Salvar também `preprocessing_params.pkl` contendo: limiares de winsorização por derivação (calculados nos folds 1–8), coeficientes dos filtros (`b`, `a` de cada filtro), e o registro das decisões tomadas (qual frequência, quais parâmetros). Este arquivo de parâmetros é essencial para aplicar exatamente o mesmo pré-processamento em dados novos.

Seção 6 — Síntese e Conexão

Tabela com número de registros que foram submetidos a cada técnica, número de registros rejeitados, estatísticas finais do dataset limpo. A pipeline de limpeza está documentada, validada e reproduzível. O próximo passo (Entregável 5) transformará esses sinais de 10 segundos em instâncias menores para extração de features.

ENTREGÁVEL 5 — Estratégia de Segmentação (Janelamento)

Arquivo: `05_segmentacao.ipynb`

Objetivo: Definir e implementar a estratégia de segmentação temporal, transformando os sinais de 10 segundos em instâncias processáveis. Validar a consistência intra- e inter-janela.

Entrada: Sinais limpos do Entregável 4; `ptb1_com_sqi.csv`

Seção 1 — Contexto e Decisão de Segmentação

Subseção 1.1 — Opções de Segmentação para ECG de 10 Segundos

Descrever as três abordagens e suas implicações:

Opção A — Registro inteiro como instância única: Cada registro de 10s é uma instância. Shape resultante: $21.837 \times 1000 \times 12$ (ou $\times 5000$ para 500 Hz). Vantagens: sem perda de contexto temporal, sem sobreposição artificial. Desvantagens: alto custo computacional para features clássicas; inadequado para análise de segmentos curtos.

Opção B — Janelas fixas sobrepostas: Cada registro é dividido em N janelas de tamanho W amostras com overlap de O amostras. Shape resultante: $(N \times 21.837) \times W \times 12$. Parâmetros recomendados: $W = 250$ amostras (2,5 segundos a 100 Hz), $O = 125$ amostras (50% de sobreposição). Justificativa fisiológica: 2,5 segundos contém em média 2–3 ciclos cardíacos (FC média ~70–80

bpm = 0.8–0.86 s/batimento), suficiente para capturar pelo menos 2 complexos QRS-T completos.

Opção C — Segmentação por batimento (beat segmentation): Detectar cada pico R e extrair janela centrada ao redor dele (ex: 200 ms antes do pico, 400 ms depois = 600 ms total = 60 amostras a 100 Hz). Ideal para análise de morfologia individual do batimento e para HRV. Desvantagem: requer detecção confiável de pico R, que pode falhar em sinais de baixa qualidade ou arritmias.

Documentar a escolha. Recomendação: implementar **Opção B** como abordagem principal (mais robusta), e **Opção C** como abordagem complementar para extração de features de morfologia e HRV no Entregável 6.

Subseção 1.2 — Propagação de Labels em Ambiente Multi-Label

Problema específico do PTB-XL: cada janela herda os labels do registro original, mas registros podem ter múltiplos labels. Definir explicitamente a estratégia de propagação: - Cada janela recebe o dicionário `scp_codes` completo do registro pai - A coluna `diagnostic_superclass` (lista de superclasses) é propagada integralmente - O `ecg_id`, `patient_id` e `strat_fold` são propagados para cada janela — garantindo que janelas do mesmo registro (e do mesmo paciente) permaneçam no mesmo fold

Seção 2 — Implementação — Opção B (Janelas Fixas Sobrepostas)

Subseção 2.1 — Função de Janelamento

Implementar `create_windows(signal, window_size, overlap, fs)` que: - Recebe: array (`n_amostras`, 12), tamanho da janela em amostras, `overlap` em amostras, `fs` - Retorna: array 3D (`n_janelas`, `window_size`, 12) - Calcula o número de janelas: `n_windows = 1 + floor((n_amostras - window_size) / (window_size - overlap))` - Para a versão com `W=250` e `O=125` a 100 Hz: cada registro de 1000 amostras gera 7 janelas não bordadas (garantir que a última janela seja descartada se incompleta — não fazer padding)

Aplicar a todos os registros e criar: - `janelas_segmentadas.npy` — shape (`N_total_janelas`, 250, 12) - `janelas_ids.csv` — DataFrame com colunas: `janela_id`, `ecg_id`, `patient_id`, `strat_fold`, `sqi_category`, `diagnostic_superclass` (string serializada da lista)

Subseção 2.2 — Verificação de Completude

Calcular o número total de janelas geradas por superclasse diagnóstica (contando multi-label: uma janela de um registro MI+CD conta para MI E para CD). Verificar que o número de janelas por fold é aproximadamente proporcional ao número de registros por fold (esperado para segmentação uniforme).

Seção 3 — Implementação — Opção C (Segmentação por Batimento)

Subseção 3.1 — Detecção de Picos R

Usar a derivação DII para detecção (maior amplitude de onda R tipicamente). Implementar o detector de pico R com `neurokit2.ecg_peaks()` ou `biosppy.signals.ecg()`, que implementam variantes do algoritmo Pan-Tompkins.

Parâmetros do Pan-Tompkins: 1. Pré-processamento interno: bandpass 5–15 Hz → derivada → quadrado → integração por janela móvel (~150 ms) 2. Threshold adaptativo baseado nos últimos 8 picos detectados 3. Janela refratária de 200 ms após cada pico (não pode haver novo pico antes disso — frequência cardíaca máxima ~300 bpm)

Validar a detecção: plotar 5 registros com picos R marcados sobre o sinal DII. Calcular o número de batimentos detectados por registro — para um ECG de 10s com FC média de 70 bpm, espera-se ~11–12 batimentos. Registros com < 5 ou > 25 batimentos detectados são suspeitos (arritmia extrema ou falha de detecção).

Subseção 3.2 — Extração das Janelas de Batimento

Para cada pico R detectado em posição `r_idx`, extrair janela de amostras `[r_idx - 20, r_idx + 40]` (200 ms antes + 400 ms depois a 100 Hz = 60 amostras totais). Descartar janelas que ultrapassem as bordas do registro.

Criar array de batimentos: `shape (N_total_batimentos, 60, 12)`. Criar DataFrame de IDs correspondente com: `beat_id`, `ecg_id`, `patient_id`, `strat_fold`, `r_peak_idx`, `rr_interval_ms` (intervalo até o próximo batimento).

Seção 4 — Validação da Segmentação

Subseção 4.1 — Estabilidade Intra-Janela (Opção B)

Para cada janela, calcular: variância das amostras (métrica de energia), número de zero crossings (ZCR) e variação total (`sum(|diff(signal)|)`). Janelas suspeitas: variância $< 0.001 \text{ mV}^2$ (sinal quase plano, possível artefato) ou variância $> 5 \text{ mV}^2$ (amplitude extrema). Calcular percentual de janelas suspeitas por derivação e por superclasse.

Subseção 4.2 — Variância Inter-Janela

Para cada registro, calcular a variância das energias de suas 7 janelas consecutivas: `var_inter = std([E_janela1, E_janela2, ..., E_janela7])`. Alta variância inter-janela indica instabilidade temporal — pode ser uma arritmia real ou um artefato. Plotar distribuição da variância inter-janela por superclasse — espera-se que AFIB tenha maior variância inter-janela (ritmo irregulares → energia variável).

Subseção 4.3 — Consistência da Segmentação Beat (Opção C)

Calcular: desvio padrão dos intervalos RR detectados por registro (SDNN preliminar). Registros com $\text{SDNN} > 150$ ms são de alta variabilidade (candidatos a AFIB ou arritmia). Verificar concordância com os labels de ritmo: registros com label AFIB devem ter SDNN maior do que SR, em média.

Seção 5 — Visualizações

Subseção 5.1 — Exemplos de Janelas por Superclasse

Montar painel: 5 colunas (uma por superclasse), 3 linhas (3 janelas de exemplo por superclasse), mostrando a derivação DII de cada janela. Anotar os labels e o `sqi_score`. Verificar que as morfologias fazem sentido visual: NORM deve ter ondas P, QRS e T bem definidas; MI deve mostrar elevação ou depressão de ST; AFIB deve mostrar ausência de onda P e irregular.

Subseção 5.2 — Alinhamento de Batimentos

Para a Opção C, plotar a sobreposição de todos os batimentos de um registro NORM (derivação DII, alinhados pelo pico R) — o resultado deve mostrar uma morfologia média bem definida com baixa dispersão, confirmando a qualidade da detecção.

Seção 6 — Síntese e Conexão

Salvar todos os arquivos de saída. Reportar: `n_janelas_total` por Opção B, `n_batimentos_total` por Opção C, distribuição por fold, percentual de janelas descartadas. Anotar qual opção será usada para qual tipo de feature no Entregável 6.

ENTREGÁVEL 6 — Extração de Atributos (Feature Extraction)

Arquivo: `06_extracao_features.ipynb`

Objetivo: Extrair representações numéricas informativas de cada janela de sinal, organizadas em quatro domínios de análise. Este é o notebook mais extenso — pode ser dividido em sub-notebooks por domínio se necessário.

Entrada: janelas do Entregável 5 (Opção B e/ou C); `janelas_ids.csv`

Seção 1 — Estratégia Geral de Extração

Subseção 1.1 — Organização por Derivação vs. Agregação

Decisão importante: calcular features por derivação (12 valores por feature) ou calcular apenas em derivações selecionadas? Recomendação: calcular em todas as 12 derivações para features simples (RMS, energia, banda de frequência), e em derivações específicas para features computacionalmente caras (entropia de amostra, DFA). Justificar no texto.

Nomear as colunas de features com o padrão `{feature_name}_{lead_name}` — ex: `rms_I`, `rms_II`, ..., `rms_V6`. Para features calculadas apenas em uma derivação, usar apenas o nome da feature.

Subseção 1.2 — Gerenciamento Computacional

Implementar extração em batch com paralelização usando `joblib.Parallel`. Estimar o tempo por janela para cada domínio. Features não lineares (entropia de amostra, DFA) são $\sim 100\times$ mais lentas que features de tempo/frequência — considerar calculá-las em subconjunto menor ou usar implementações otimizadas (`EntroPy`, `antropy`).

Nota sobre métodos previstos no pipeline e não implementados aqui:

- **Hilbert-Huang Transform (HHT/EMD):** o pipeline de biossinais lista HHT como método de domínio tempo-frequência. Optou-se por não implementá-lo neste projeto pelas seguintes razões: (a) a Empirical Mode Decomposition (EMD), base do HHT, é sensível a ruído de borda e requer ajuste cuidadoso do número de IMFs — complexidade de implementação desproporcional ao ganho esperado; (b) HHT tem uso consolidado principalmente em sinais não-estacionários de longa duração; com janelas de 2,5 s e registros de 10 s, a STFT e a DWT fornecem resolução tempo-frequência suficiente para ECG diagnóstico. Documentar esta decisão no relatório.
- **Hjorth Parameters:** listados no pipeline com a indicação “(EEG)”, o que reflete sua origem e uso mais consolidado em eletroencefalografia. Para ECG de 12 derivações, os parâmetros de Hjorth (Activity, Mobility, Complexity) capturam informação parcialmente redundante com as features de RMS, ZCR e variância já incluídas neste notebook. Optou-se por não incluí-los para manter o conjunto de features interpretável e evitar redundância desnecessária antes do Entregável 9. Documentar esta decisão no relatório.

Seção 2 — Features de Domínio do Tempo (Janelas Opção B: 250 amostras $\times$ 12 derivações)

Subseção 2.1 — Estatísticas Básicas de Amplitude

Para cada janela e cada derivação calcular: - **Mean (μ)**: média aritmética das amostras — idealmente próxima de 0 após filtragem passa-alta - **RMS**: raiz da média dos quadrados — $\text{sqrt}(\text{mean}(x^2))$ — representa a magnitude energética do sinal - **MAV (Mean Absolute Value)**: $\text{mean}(|x|)$ — menos sensível a outliers que RMS - **Variância**: $\text{var}(x)$ — dispersion ao redor da média - **Desvio Padrão**: $\text{std}(x)$ - **Amplitude Pico-a-Pico**: $\text{max}(x) - \text{min}(x)$ — detecta modulações de amplitude - **Skewness e Kurtosis**: da amplitude (diferentes das do SQI — aqui são features, não métricas de qualidade)

Total: 7 features $\times$ 12 derivações = 84 features

Subseção 2.2 — Zero Crossing Rate (ZCR)

$\text{ZCR} = \text{sum}(\text{sign}(x[1:]) \neq \text{sign}(x[:-1])) / (\text{len}(x) - 1)$ — proporção de mudanças de sinal por amostra. Em ECG, ZCR alta em uma derivação pode indicar ruído de alta frequência residual ou QRS fragmentado. Calcular por derivação. Total: 12 features.

Subseção 2.3 — Features de Morfologia ECG (derivações DII, V5)

Para estas features, trabalhar sobre as janelas de batimento (Opção C), que têm os batimentos alinhados pelo pico R. Para cada batimento:

- **Duração do QRS**: número de amostras desde o início (onda Q) até o final (fim da onda S), em milissegundos. O início e fim do QRS podem ser detectados como os pontos onde a amplitude cruza 10% da amplitude máxima do complexo.
- **Amplitude do pico R**: valor máximo do sinal na janela de batimento (confirmado como pico R)
- **Profundidade do pico Q**: valor mínimo nas 30 ms antes do pico R (se negativo, é onda Q)
- **Amplitude do segmento ST**: valor médio do sinal nas 80 ms após o fim do QRS — elevação ou depressão de ST é crítica para diagnóstico de infarto
- **Intervalo QT**: do início do QRS ao fim da onda T. Prolongado no diagnóstico LNGQT.
- **Amplitude máxima da onda T**: valor de pico da onda T (janela de 200–400 ms após o pico R)

Para obter um valor por janela de 2,5s, usar a mediana sobre os ~2–3 batimentos presentes na janela.

Total: 6 features $\times$ 2 derivações = 12 features (+ podem ser expandidas para todas as 12 derivações nas features clinicamente relevantes)

Subseção 2.4 — Features de HRV (Heart Rate Variability) no Domínio do Tempo

HRV é calculada a partir da série de intervalos RR (tempo entre batimentos consecutivos). A partir dos picos R detectados na Opção C, extrair para cada registro completo de 10s:

- **meanRR**: intervalo RR médio em milissegundos = $60.000 / \text{FC_média}$ (FC em bpm)
- **sdRR (SDNN)**: desvio padrão dos intervalos RR — marcador global de variabilidade
- **RMSSD**: $\sqrt{\text{mean}(\text{diff}(\text{RR})^2)}$ — raiz da média dos quadrados das diferenças sucessivas de RR — marcador de atividade parassimpática de curto prazo
- **pNN50**: proporção de pares de batimentos consecutivos com diferença > 50 ms — também marcador parassimpático
- **CV_RR (Coefficient of Variation)**: $\text{sdRR} / \text{meanRR}$ — variabilidade normalizada pela frequência cardíaca média

Nota: 10 segundos de ECG fornece apenas ~10–12 batimentos, o que é curto para HRV robusta (análise padrão usa 5 min). Documentar esta limitação. As features de HRV do PTB-XL têm validade qualitativa, não quantitativa clínica.

Total: 5 features (no nível do registro, propagadas para todas as janelas do mesmo registro)

Seção 3 — Features de Domínio da Frequência

Subseção 3.1 — Estimativa da PSD via Welch

Para cada janela de 250 amostras (2.5 s a 100 Hz), calcular a PSD com `scipy.signal.welch`: - `nperseg` = 100 (janela de 1s para boa resolução em frequência: resolução de 1 Hz) - `noverlap` = 50 (50% de sobreposição) - `window` = 'hann' (janela de Hann — boa supressão de lóbulos laterais) - `nfft` = 256 (zero-padding para melhor resolução espectral visual) - Resultado: array de frequências (0–50 Hz) e PSD correspondente (em mV^2/Hz)

Calcular por derivação. Para visualização, plotar o espectro médio de 50 janelas por superclasse (derivação DII), com bandas de frequência sombreadas.

Subseção 3.2 — Potência por Banda de Frequência

Integrar a PSD (usando regra trapezoidal `numpy.trapz`) nas seguintes bandas:

Para análise morfológica do ECG (calculadas por janela de 2,5 s e por derivação):

- **Banda P (0.5–5 Hz)**: energia da onda P e componentes de baixa frequência
- **Banda QRS (5–25 Hz)**: energia do complexo QRS — o mais energético
- **Banda T (0.5–5 Hz, mesma faixa de P)**: energia da onda T — nota: P e T se sobrepõem spectralmente; a distinção temporal é feita pelo domínio do tempo
- **Banda total (0.5–40 Hz)**: energia total do sinal filtrado

Total de features morfológicas: 4 bandas $\times$ 12 derivações = 48 features

Para análise de HRV espectral (calculadas **exclusivamente no nível do registro completo de 10 s**, nunca nas janelas de 2,5 s): - **LF (0.04–0.15 Hz)**: Low Frequency — modulação simpática e parassimpática - **HF (0.15–0.4 Hz)**:

High Frequency — modulação parassimpática (respiratória) - **Razão LF/HF:** balanço autonômico simpato-vagal

Importante — por que não calcular HRV espectral nas janelas de 2,5 s: a resolução espectral mínima de uma janela de duração T é $1/T$ Hz. Para $T = 2,5$ s, isso resulta em resolução de 0,4 Hz — exatamente o limite superior da banda HF. As bandas LF (0,04–0,15 Hz) e HF (0,15–0,4 Hz) estão inteiramente abaixo dessa resolução e não são fisicamente resolvíveis em janelas de 2,5 s. Calcular essas bandas sobre janelas curtas produziria valores numericamente espúrios sem significado fisiológico. Por isso, as features de HRV espectral são calculadas sobre o registro inteiro de 10 s (via `scipy.signal.welch` aplicado à série de intervalos RR detectados pela segmentação beat), e então propagadas para todas as janelas do mesmo registro como valor constante. Documentar esta limitação no relatório: 10 s é curto para HRV espectral robusta clinicamente (padrão clínico usa 5 min), mas fornece uma estimativa qualitativa válida para fins de classificação.

Total de features de HRV espectral: 3 features no nível de registro, propagadas às janelas.

Subseção 3.3 — Features Espectrais Descritivas

Para cada janela e derivação: - **Frequência de Pico Espectral:** frequência da maior amplitude na PSD — deve ser próxima da frequência do complexo QRS (~5–20 Hz para ECG normal) - **Frequência Mediana Espectral:** frequência que divide a PSD em metades iguais de energia - **Frequência Média Ponderada:** $\text{sum}(\text{freq} * \text{PSD}) / \text{sum}(\text{PSD})$ — centróide espectral - **Largura de Banda Efetiva:** frequência onde a PSD cai a -3 dB do pico

Total: 4 features $\times$ 12 derivações = 48 features

Seção 4 — Features de Domínio Tempo-Frequência

Subseção 4.1 — Short-Time Fourier Transform (STFT)

Para cada janela de 250 amostras, calcular STFT: `scipy.signal.stft(signal, fs=100, nperseg=50, noverlap=25)` — janela deslizante de 50 amostras (0.5s) com 50% de sobreposição. Resultado: espectrograma de shape (freqs, tempo_frames) por derivação.

Extrair do espectrograma: - **Potência média por banda em cada frame temporal:** média temporal da potência nas bandas QRS e T — captura dinâmica temporal da energia - **Centróide espectral médio por frame:** evolução temporal do centróide - **Variabilidade temporal do espectrograma:** desvio padrão da potência total ao longo dos frames — alta variabilidade indica ritmo irregular

Para visualização: plotar espectrogramas médios de 5 registros (NORM, MI, AFIB, LBBB, HYP) na derivação DII como heatmaps (eixo x = tempo, eixo y = frequência, cor = potência em dB).

Total: ~ 10 features $\times$ 2 derivações (DII, V5) = ~ 20 features

Subseção 4.2 — Transformada Wavelet Discreta (DWT)

Justificativa da escolha da wavelet: a wavelet de Daubechies de ordem 4 (db4) tem forma morfológica similar ao complexo QRS, tornando-a especialmente sensível à detecção e análise de QRS. A wavelet Symlet 8 (sym8) é mais simétrica e adequada para análise de onda T e P. Usar **db4** para features focadas em QRS; **sym8** para features gerais.

Aplicar DWT com 4 níveis de decomposição usando `pywt.wavedec(signal, 'db4', level=4)`. Para cada registro a 100 Hz, os coeficientes de detalhe correspondem às bandas: - **cD1 (nível 1)**: 25–50 Hz — ruído residual (deve ter energia baixa após filtragem) - **cD2 (nível 2)**: 12.5–25 Hz — energia do complexo QRS (conteúdo principal) - **cD3 (nível 3)**: 6.25–12.5 Hz — energia da onda T e QRS lento - **cD4 (nível 4)**: 3.125–6.25 Hz — energia de ondas P e T lentas - **cA4 (aproximação)**: 0–3.125 Hz — componentes de muito baixa frequência (drift residual)

Extraír de cada coeficiente: energia (`sum(cD**2)`), RMS (`sqrt(mean(cD**2))`), entropia de Shannon dos quadrados. Calcular para a derivação DII e V5.

Total: 5 níveis $\times$ 3 features $\times$ 2 derivações = 30 features

Seção 5 — Features Não Lineares

Subseção 5.1 — Entropia de Amostra (Sample Entropy)

Sample Entropy (SampEn) mede a irregularidade do sinal: probabilidade de que padrões semelhantes em m amostras permaneçam semelhantes em $m+1$ amostras. Valor baixo = sinal previsível/regular; valor alto = sinal irregular/complexo.

Parâmetros: $m = 2$ (comprimento do padrão), $r = 0.2 \times \text{std}(\text{signal})$ (tolerância de similaridade — 20% do desvio padrão é o padrão da literatura de HRV). Usar a biblioteca `antropy` para implementação eficiente: `antropy.sample_entropy(signal, order=2, metric='chebyshev')`.

Calcular para a série RR (HRV) e para o sinal bruto de DII por janela. Incluir também **Approximate Entropy (ApEn)** (`antropy.app_entropy`) para comparação — ApEn é mais rápida mas enviesada para amostras curtas.

Total: 2 features (SampEn e ApEn, calculados para DII e para série RR)

Subseção 5.2 — Detrended Fluctuation Analysis (DFA)

DFA calcula o expoente de escala que caracteriza as autocorrelações de longo alcance do sinal. Implementar com `antropy.detrended_fluctuation(signal)`: 1. Integrar o sinal (cumulative sum após remover a média) 2. Dividir em janelas de tamanho n 3. Remover tendência linear em cada janela (detrend) 4. Calcular RMS dos resíduos $F(n)$ 5. Fazer regressão log-log de $F(n)$ vs. n para obter

Interpretar para ECG: 0.5 = ruído branco (sem correlações); 1.0 = fractal saudável ($1/f$ noise, típico de ECG normal); > 1.5 = processo Browniano (sinal com correlações de longo prazo excessivas, pode indicar patologia rígida). Espera-se que NORM tenha mais próximo de 1.0 que as classes patológicas.

Calcular para a série RR e para o sinal DII. Total: 2 features.

Subseção 5.3 — Dimensão Fractal (Higuchi)

`antropy.higuchi_fd(signal, kmax=10)` — calcula a dimensão fractal D . ECG normal: $D \sim 1.2$ – 1.6 . Sinal altamente periódico (pouco complexo): $D \sim 1.0$. Sinal totalmente aleatório: $D \sim 2.0$. Calcular para DII. Total: 1 feature.

Subseção 5.4 — Análise do Plot de Poincaré

Para cada registro, a partir da série de intervalos RR: - **SD1**: desvio padrão das diferenças consecutivas de RR no eixo perpendicular à identidade do gráfico de Poincaré. $SD1 = \sqrt{0.5 * \text{var}(\text{diff}(RR))}$. Reflete variabilidade de curto prazo — dominada pelo sistema parassimpático. - **SD2**: desvio padrão ao longo do eixo da identidade. $SD2 = \sqrt{2 * \text{var}(RR) - 0.5 * \text{var}(\text{diff}(RR))}$. Reflete variabilidade de longo prazo — balanço simpato-vagal. - **Razão SD1/SD2**: equilíbrio entre as duas formas de variabilidade

Plotar o gráfico de Poincaré (scatter de RR_n vs. RR_{n+1}) para 3 registros NORM e 3 AFIB. Registros NORM devem mostrar elipse compacta orientada diagonalmente; AFIB deve mostrar nuvem dispersa sem estrutura elíptica clara.

Total: 3 features (SD1, SD2, razão)

Seção 6 — Consolidação do Dataset de Features

Subseção 6.1 — Montagem do DataFrame

Concatenar todas as features calculadas em um único DataFrame. Estrutura de colunas: - Colunas de identificação: `janela_id`, `ecg_id`, `patient_id`, `strat_fold`, `sqi_category` - Colunas de label: `diagnostic_superclass` (lista serializada), `n_superclasses`, colunas binárias de presença de cada superclasse: `label_NORM`, `label_MI`, `label_CD`, `label_HYP`, `label_STTC` - Colunas de features: todas as features calculadas, nomeadas com o padrão `{domínio}_{feature}_{derivação}`

Verificar: ausência de NaN em todas as linhas de features (NaN indica falha de cálculo — investigar e corrigir). Calcular número total de features extraídas.

Subseção 6.2 — Tabela Descritiva das Features

Criar `features_raw_summary.csv` com: nome da feature, domínio (tempo/frequência/tempo-frequência/não-linear), derivação, média global, desvio padrão global, média por superclasse (5 colunas), coeficiente de variação. Esta tabela servirá como referência no relatório final.

Subseção 6.3 — Visualizações Exemplificativas

Para cada domínio, criar uma figura ilustrativa: - **Tempo**: boxplots do RMS por derivação, separados por superclasse - **Frequência**: curvas de PSD média por superclasse (derivação DII, escala log) - **Wavelet**: barras de energia por nível de decomposição, agrupadas por superclasse - **Não linear**: scatter plot de SD1 vs. SD2 com pontos coloridos por superclasse; scatter de SampEn vs. DFA

Seção 7 — Síntese e Conexão

Salvar `features_raw.parquet` (formato parquet para eficiência com datasets largos). Reportar: número total de features, dimensão do dataset ($N_{\text{janelas}} \times N_{\text{features}}$), memória utilizada. A próxima etapa (Entregável 7) combinará e normalizará estas features.

ENTREGÁVEL 7 — Engenharia de Features

Arquivo: `07_engenharia_features.ipynb`

Objetivo: Criar features de segunda ordem, normalizar o dataset e analisar relevância e redundância. Este entregável transforma o dataset bruto de features em um dataset refinado, pronto para redução de dimensionalidade.

Entrada: `features_raw.parquet`

Seção 1 — Diagnóstico do Dataset de Features

Subseção 1.1 — Verificação de Integridade

Carregar `features_raw.parquet`. Calcular: - `df.isnull().sum()` — número de NaN por feature; features com $> 5\%$ de NaN serão removidas neste entregável - `df.std()` — desvio padrão por feature; features com $\text{std} < 1e-6$ são constantes e serão removidas - `df.describe()` — verificar ranges suspeitos (valores negativos onde não deveria haver, etc.)

Criar lista `features_to_remove_zero_variance` e `features_to_remove_high_nan`. Remover e documentar.

Subseção 1.2 — Tratamento de NaN Remanescentes

Para features com $\text{NaN} < 5\%$: imputar com a mediana calculada nos folds de treino (1–8). Não usar a média — é sensível a outliers. Implementar: calcular medianas por feature e por superclasse no conjunto de treino; para NaN em treino, usar mediana da mesma superclasse; para NaN em val/teste, usar a mediana de treino da superclasse correspondente (se conhecida) ou a mediana global de treino.

Seção 2 — Features Derivadas e Combinadas

Subseção 2.1 — Razões entre Features Espectrais

Criar features de razão entre bandas de frequência — capturando o equilíbrio relativo de energia: - `ratio_QRS_T_{lead}` = `power_band_QRS_{lead}` / `power_band_T_{lead}` — razão entre energia do QRS e da onda T; relevante para diagnóstico de alterações de repolarização - `ratio_LF_HF` = `power_LF` / `power_HF` — balanço autonômico (HRV espectral) - `ratio_D2_D3_wavelet_{lead}` = `wavelet_energy_D2_{lead}` / `wavelet_energy_D3_{lead}` — concentração de energia wavelet nas bandas de QRS vs. T

Justificar cada razão criada com base no que ela captura fisiologicamente.

Subseção 2.2 — Features de Delta (Primeira Diferença Temporal)

Para features que variam ao longo das janelas consecutivas de um mesmo registro, calcular a primeira diferença (tendência): `delta_{feature}` = `feature_janela_k` - `feature_janela_{k-1}` (para $k > 0$ dentro do mesmo registro)

Aplicar para: RMS por derivação, potência de banda QRS, meanRR. Janelas que são a primeira do registro têm $\text{delta} = 0$ (ou NaN e imputar com 0). Estas features capturam dinâmica temporal intrarregistro — ex: frequência cardíaca acelerando ou desacelerando.

Subseção 2.3 — Normalização por Linha de Base NORM

Para features de amplitude (RMS, energia, potência espectral), criar versão normalizada pela mediana da classe NORM no conjunto de treino: `feature_normalized` = `feature` / `median_NORM_train`. O resultado é adimensional e expressa o desvio em relação ao padrão normal — interpretável clinicamente como “quantas vezes maior que o normal”.

Subseção 2.4 — Agregações Intra-Registro

Para cada registro, agregar as features de suas janelas usando: `mean`, `std`, `min`, `max`, `p5`, `p25`, `p75`, `p95`. Criar um DataFrame no nível de registro (uma linha por `ecg_id`), que será o input para modelos que operam no nível de registro (ao invés de janela). Salvar como `features_registro.parquet`. Manter também o DataFrame no nível de janela para modelos que operam sobre sequências.

Seção 3 — Normalização do Dataset

Subseção 3.1 — Análise de Distribuições e Transformações

Antes de normalizar, verificar a distribuição de cada feature: - Features com distribuição fortemente assimétrica à direita ($\text{skewness} > 2$): testar transformação log (se todos os valores forem positivos) ou Box-Cox - Features com outliers extremos: a winsorização feita nos sinais no Entregável 4 deve ter reduzido, mas verificar se features como kurtosis ou DFA ainda têm outliers extremos

Apresentar: histogramas das 10 features com maior skewness antes e depois de transformação, escolhendo transformação apenas se melhora substancialmente (reduz $|\text{skewness}|$ em $> 50\%$).

Subseção 3.2 — Normalização Robusta (RobustScaler)

Aplicar RobustScaler: `feature_scaled = (feature - median_train) / IQR_train`. Esta normalização usa mediana e IQR (ao invés de média e std do StandardScaler), sendo robusta a outliers remanescentes.

Regra crítica de data leakage: calcular `median_train` e `IQR_train` **exclusivamente** com os dados dos folds 1–8. Serializar o scaler ajustado como `scaler_params.pkl`. Aplicar o scaler (sem re-ajustar) aos folds 9 e 10.

Após normalização, cada feature deve ter mediana 0 e IQR 1 nos dados de treino. Verificar com `df_treino[feature_cols].describe()`.

Seção 4 — Validação das Features

Subseção 4.1 — Relevância por Correlação com Variável Resposta

Para o problema multi-classe multi-label, calcular relevância de cada feature para cada superclasse usando ANOVA F-statistic entre janelas com label da classe = 1 vs. label = 0. `sklearn.feature_selection.f_classif` retorna F e p-valor por feature. Fazer para cada uma das 5 superclasses separadamente.

Criar tabela com: feature, F_NORM, F_MI, F_CD, F_HYP, F_STTC, max_F (a maior F entre as 5 classes — indica a classe para qual a feature é mais discriminativa). Plotar ranking das top-30 features por max_F.

Subseção 4.2 — Análise de Redundância

Calcular a matriz de correlação de Spearman entre todas as features do conjunto de treino. Identificar pares com $|r| > 0.90$ como altamente redundantes. Criar um grafo onde cada nó é uma feature e cada aresta conecta pares com $|r| > 0.90$. Aplicar um algoritmo guloso para identificar um conjunto mínimo de features que quebra todas as arestas (manter a feature com maior max_F de cada par

redundante, remover a outra). Documentar quais features foram identificadas como redundantes — a remoção final ocorrerá no Entregável 9.

Seção 5 — Síntese e Conexão

Salvar `features_engineered.parquet` (com todas as features originais + derivadas + normalizadas, sem remover redundantes ainda — isso é feito no Entregável 9). Salvar `scaler_params.pkl`. Reportar: número de features antes e depois da engenharia, número de features flagadas como redundantes, número de features com alta relevância.

ENTREGÁVEL 8 — Redução de Dimensionalidade

Arquivo: `08_reducao_dimensionalidade.ipynb`

Objetivo: Reduzir o espaço de features para um conjunto de componentes não correlacionados que retém a maior parte da variância, facilitando visualização, reduzindo custo computacional e potencialmente revelando estrutura latente.

Entrada: `features_engineered.parquet`

Seção 1 — Motivação

Subseção 1.1 — Dimensionalidade e Maldição da Dimensionalidade

Calcular a dimensionalidade atual (número de features após Entregável 7). Para algoritmos baseados em distância (kNN, SVM com kernel RBF), alta dimensionalidade degrada a discriminabilidade porque a distância entre pontos converge para valores similares em espaços de muitas dimensões (fenômeno conhecido como “concentração de distâncias”). Dimensionalidade muito maior que $\sqrt{n_samples}$ é suspeita — calcular essa relação e discutir.

Seção 2 — PCA (Principal Component Analysis)

Subseção 2.1 — Padronização Prévia ao PCA

PCA é sensível à escala — features com variância maior dominam os primeiros componentes. Como o dataset já foi normalizado com `RobustScaler` no Entregável 7, verificar se os desvios padrão estão aproximadamente iguais. Se necessário, aplicar uma normalização adicional para garantir variância unitária: `sklearn.preprocessing.StandardScaler`. Ajustar apenas nos folds de treino.

Subseção 2.2 — Ajuste e Análise do PCA

Ajustar `sklearn.decomposition.PCA(n_components=None, random_state=42)` nos folds de treino. Isso calcula todos os componentes possíveis.

Calcular e apresentar:

- **Variância explicada por componente:** `pca.explained_variance_ratio_`
- **Scree plot:** gráfico de linha com componentes no eixo x e variância explicada (%) no eixo y. Marcar o ponto de “cotovelo” — onde a curva começa a se tornar aproximadamente plana
- **Variância acumulada:** plot da variância acumulada; marcar as linhas horizontais em 80%, 90%, 95% e 99%; anotar o número de componentes necessário para atingir cada limiar
- **Tabela:** n_componentes para 80%, 90%, 95% e 99% de variância retida

Subseção 2.3 — Análise dos Loadings dos Primeiros 10 PCs

Para os 10 primeiros componentes principais, criar um heatmap de loadings: eixo x = componentes PC1–PC10, eixo y = features originais (ordenadas por cluster de domínio). Usar colormap divergente (-1 a +1). Identificar, para cada componente, as 5 features com maior loading absoluto e descrevê-las em texto.

Por exemplo: se PC1 tem altos loadings positivos em `rms_I`, `rms_II`, `rms_III` e alto loading negativo em `wavelet_energy_D1_V1`, interpretar como “PC1 captura a energia global de amplitude nas derivações de membros em contraste com o ruído de alta frequência em V1.”

Subseção 2.4 — Visualização no Espaço dos 2 Primeiros PCs

Scatter plot de PC1 vs. PC2, com pontos coloridos por superclasse diagnóstica. Para o problema multi-label, colorir cada ponto pela sua superclasse de maior likelihood. Plotar elipses de confiança (1 desvio padrão) por superclasse. Calcular o overlap visual entre classes — alta sobreposição indica que os 2 primeiros PCs não separam bem as classes (esperado; serão necessários mais componentes para o RP).

Repetir para PC1 vs. PC3 e PC2 vs. PC3 — verificar se alguma combinação separa melhor.

Subseção 2.5 — Escolha do Número de Componentes

Documentar a decisão: escolher o número de componentes que retém 95% da variância como padrão. Discutir alternativas: usar o critério Kaiser (reter componentes com autovalor > 1) ou usar a análise do cotovelo. Justificar a escolha.

Aplicar a transformação PCA com o número escolhido: `pca_final = PCA(n_components=k_95, random_state=42).fit(X_treino)`. Salvar o modelo como `pca_model.pkl`. Transformar treino, val e teste separadamente.

Seção 3 — ICA (Independent Component Analysis)

Subseção 3.1 — Motivação e Implementação

Enquanto PCA maximiza variância e produz componentes ortogonais (não correlacionados), ICA maximiza independência estatística entre os componentes — uma condição mais forte. Para ECG, ICA pode separar contribuições de diferentes câmaras cardíacas ou identificar componentes de ruído residual.

Aplicar `sklearn.decomposition.FastICA(n_components=20, random_state=42, max_iter=500)` aos dados de treino no espaço PCA (é comum aplicar ICA após PCA para reduzir dimensionalidade e estabilizar o algoritmo).

Subseção 3.2 — Inspeção dos Componentes Independentes

Para cada componente ICA, calcular seu espectro de frequência (se fizer sentido para as features mistas) e suas correlações com as features originais. Identificar se algum componente tem características de artefato: correlação alta com métricas de qualidade baixa, ou espectro concentrado em 50 Hz. Se identificado, este componente pode ser descartado.

Seção 4 — Síntese e Conexão

Salvar `features_pca.parquet` com os scores dos componentes PCA para todas as amostras. Salvar `pca_model.pkl`. Reportar: dimensionalidade antes e depois, variância retida, tempo de transformação. Os dados reduzidos pelo PCA e o dataset de features engineered original serão ambos mantidos — a seleção de atributos (Entregável 9) será feita sobre o dataset de features originais (não sobre os PCs, que são combinações lineares e perdem interpretabilidade individual).

ENTREGÁVEL 9 — Seleção de Atributos

Arquivo: `09_selecao_atributos.ipynb`

Objetivo: Identificar o subconjunto mínimo de features originais interpretáveis com máximo poder discriminativo para a tarefa de classificação multi-label. Diferente do PCA (que cria novas dimensões), a seleção mantém features originais com significado fisiológico.

Entrada: `features_engineered.parquet`; resultados de relevância do Entregável 7

Seção 1 — Definição da Tarefa de Seleção

Subseção 1.1 — Multi-Label vs. Multi-Classe

Discutir o problema de seleção de features em contexto multi-label: a relevância de uma feature pode diferir entre classes. Feature X pode ser altamente discriminativa para MI vs. NORM mas irrelevante para CD vs. NORM.

Estratégia adotada: realizar seleção separada para cada superclasse (problema binário one-vs-rest), depois unir os conjuntos selecionados. Uma feature é incluída se for selecionada por ao menos 2 dos 5 classificadores binários.

Seção 2 — Métodos Filter

Subseção 2.1 — ANOVA F-statistic

Reutilizar os resultados do Entregável 7 (F-statistic por feature por superclasse). Para cada superclasse, selecionar top-N features com maior F e $p < 0.05$ após correção de Bonferroni (multiplicar p-valor pelo número de features testadas). Usar $N = 50$ como ponto de partida. Calcular a união das features selecionadas por todas as 5 superclasses.

Subseção 2.2 — Mutual Information

Usar `sklearn.feature_selection.mutual_info_classif` para cada problema binário (one-vs-rest por superclasse). A MI é calculada sem assumir linearidade. Selecionar top-50 por superclasse. Calcular a união e compará-la com a seleção por ANOVA. Features que aparecem em ambas as listas têm robustez metodológica.

Subseção 2.3 — ReliefF

Implementar com `scikit-rebate` ou manualmente: para cada amostra, encontrar os $k=10$ vizinhos mais próximos da mesma classe (“near-hits”) e das outras classes (“near-misses”); atualizar o peso de cada feature baseado em suas diferenças nos vizinhos. Features que distinguem classes diferentes recebem peso positivo; features que variam dentro da mesma classe recebem peso negativo. Usar subset estratificado de 5.000 amostras do treino para viabilidade computacional.

Subseção 2.4 — Síntese Filter: Ranking Unificado

Para cada feature, criar um score de consenso filter: `score_filter = (rank_ANOVA + rank_MI + rank_ReliefF) / 3` (rank normalizado de 0 a 1, onde 1 = mais relevante). Plotar o histograma deste score — features com `score > 0.7` são o conjunto “Filter Top”.

Seção 3 — Métodos Wrapper

Subseção 3.1 — Sequential Forward Selection (SFS)

Usar `mlxtend.feature_selection.SequentialFeatureSelector` com `forward=True`, `k_features='best'`, `scoring='balanced_accuracy'`, `cv=5` (usando apenas os folds de treino 1–8). Classificador proxy: `sklearn.tree.DecisionTreeClassifier(max_depth=5, random_state=42)` — rápido e sensível a interações entre features.

Aplicar sobre as features do conjunto “Filter Top” (ao invés de todas as features — reduz custo computacional drasticamente). Plotar a curva de `balanced_accuracy` vs. número de features — o platô indica o ponto ótimo. Seleccionar o número mínimo de features que atinge 95% da accuracy máxima.

Subseção 3.2 — Sequential Backward Elimination (SBE)

Aplicar `SequentialFeatureSelector` com `forward=False`, partindo do conjunto “Filter Top” e removendo features uma a uma. Comparar o resultado com SFS. Se SFS e SBE convergem para um conjunto similar, há confiança no resultado. Se divergem significativamente, há interações complexas e o conjunto maior (união de SFS e SBE) deve ser preferido.

Seção 4 — Método Embedded

Subseção 4.1 — LASSO (Regularização L1)

Usar `sklearn.linear_model.LogisticRegression(penalty='l1', solver='liblinear', random_state=42)` para cada problema binário one-vs-rest. Varrer `C` (inverso de `lambda`) em escala logarítmica: `C_range = np.logspace(-4, 2, 50)`. Para cada valor de `C`, ajustar o modelo nos folds de treino e registrar quais features têm coeficiente > 0 .

Plotar o caminho de regularização: eixo $x = \log(C)$, eixo y = coeficientes por feature. Features cujos coeficientes permanecem não-zero mesmo para valores pequenos de `C` são as mais robustas.

Para a seleção final embedded: usar o valor de `C` que maximiza a `balanced_accuracy` no fold 9 (validação). Coletar as features com coeficiente > 0 nesse `C` ótimo para cada superclasse, depois unir.

Seção 5 — Validação Estatística das Features Candidatas

Subseção 5.1 — Definição do Conjunto Candidato

União das features selecionadas por: consenso filter (`score > 0.7`), SFS/SBE (top features), e LASSO (coeficiente não zero para ao menos 2 superclasses). Este é o conjunto candidato, que ainda pode ser grande.

Subseção 5.2 — Testes de Hipótese por Feature

Para cada feature no conjunto candidato, realizar testes de hipótese comparando as distribuições entre os pares de classes relevantes: - Como a maioria das distribuições não é normal (confirmado no Entregável 3), usar **Mann-Whitney U** (`scipy.stats.mannwhitneyu`) para comparações binárias e **Kruskal-Wallis** (`scipy.stats.kruskal`) para comparações multi-grupo - Realizar todos os pares: NORM vs. MI, NORM vs. CD, NORM vs. HYP, NORM vs. STTC, MI vs. CD, MI vs. HYP, MI vs. STTC, CD vs. HYP, CD vs. STTC, HYP vs. STTC — total de 10 comparações por feature

Subseção 5.3 — Correção de Múltiplas Comparações

O número total de comparações = $n_features_candidatas \times 10$ pares. Sem correção, a taxa de falsos positivos explodiria. Aplicar dois métodos: - **Bonferroni**: $p_corrigido = p_original \times n_comparações_totais$. Muito conservador — muitas features serão rejeitadas. - **Benjamini-Hochberg (FDR)**: controla a taxa esperada de falsos positivos ($q = 0.05$). Menos conservador que Bonferroni. Usar `statsmodels.stats.multitest.multipletests(pvalues, method='fdr_bh')`.

Reportar o número de features que sobrevivem à correção por cada método. Usar FDR como critério principal (mais adequado para exploração de dados); Bonferroni como verificação de robustez.

Subseção 5.4 — Effect Size

Para cada comparação par-a-par significativa (após correção FDR), calcular: - **Rank-Biserial Correlation r**: para Mann-Whitney: $r = 1 - (2*U) / (n1*n2)$, varia de -1 a 1. $|r| < 0.1$ = trivial; 0.1–0.3 = pequeno; 0.3–0.5 = médio; > 0.5 = grande. - Descartar comparações com $|r| < 0.1$ mesmo que o p-valor seja significativo (grande amostra pode tornar trivialidades significativas).

Seção 6 — Conjunto Final de Features

Subseção 6.1 — Decisão Final

Criar tabela-síntese:

Feature	Domínio	Score		Wrapper?	LASSO?	p FDR	Effect size	Incluída?
		Filter					max	
rms_DII	tempo	0.85	Sim	Sim	<	0.001	0.62	Sim
...	...	...	...	...	...	...	...	...

Incluir feature se: $score_filter > 0.5$ **OU** (wrapper = Sim **E** LASSO = Sim), **E** effect size máximo > 0.1 .

Subseção 6.2 — Validação Fisiológica do Conjunto Final

Escrever um parágrafo argumentando a coerência clínica das features selecionadas. Features de HRV (RMSSD, pNN50, SD1, SD2) são esperadas — são marcadores estabelecidos de risco cardiovascular. Features de morfologia QRS (duração, amplitude R, ST) são esperadas — são a base do diagnóstico clínico de ECG. Features de energia espectral na banda QRS são esperadas — capturam a “força” do complexo ventricular. A presença dessas features valida que o processo de seleção encontrou features fisiologicamente relevantes, e não correlações espúrias.

Subseção 6.3 — Verificação de Remoção de Redundância

Calcular VIF preliminar (usando `statsmodels.stats.outliers_influence.variance_inflation_factor`) para o conjunto selecionado. Verificar que não há $VIF > 10$. Se houver, remover iterativamente a feature de menor relevância até todos estarem abaixo de 10. (VIF será calculado completamente no Entregável 10.)

Seção 7 — Síntese e Conexão

Salvar `features_final.parquet` com apenas as features selecionadas (+ colunas de identificação e labels). Salvar `feature_selection_report.csv` com a tabela completa de seleção. Reportar: `n_features_candidatas`, `n_features_finais`, redução percentual de dimensionalidade.

ENTREGÁVEL 10 — Validação Estatística Final do Dataset (Pronto para RP)

Arquivo: `10_validacao_final.ipynb`

Objetivo: Verificação completa e sistemática de todas as propriedades que o dataset precisa ter para ser submetido a algoritmos de Reconhecimento de Padrões. Este é o “checklist de saída” do pipeline — cada item verificado e documentado.

Entrada: `features_final.parquet`

Seção 1 — Análise de Multicolinearidade (VIF)

Subseção 1.1 — Cálculo do VIF

O Variance Inflation Factor para a feature i é: $VIF_i = 1 / (1 - R^2_i)$, onde R^2_i é o R^2 da regressão de `feature_i` sobre todas as outras features. $VIF = 1 \rightarrow$ sem multicolinearidade; $VIF = 5 \rightarrow$ moderada; $VIF > 10 \rightarrow$ severa (a feature i pode ser predita com 90% de precisão pelas demais $\rightarrow$ redundância grave).

Usar `statsmodels.stats.outliers_influence.variance_inflation_factor(X_treino, i)` para cada feature i , onde `X_treino` é a matriz de features dos folds 1–8 com uma coluna constante adicionada.

Apresentar tabela ordenada por VIF (maior primeiro). Criar gráfico de barras horizontais.

Subseção 1.2 — Iteração de Remoção

Enquanto $\max(\text{VIF}) > 10$: 1. Identificar a feature com maior VIF 2. Calcular seu F-statistic de relevância (do Entregável 9) 3. Remover a feature com VIF mais alto **somente** se sua remoção não comprometer a representação do domínio — ou seja, se outra feature do mesmo domínio com $\text{VIF} < 10$ cobrir informação similar 4. Recalcular VIF para o conjunto restante 5. Registrar cada remoção em uma tabela de log

Meta: conjunto final com todos os VIFs < 5 (critério conservador).

Seção 2 — Análise de Separabilidade Estatística

Subseção 2.1 — Curvas de Densidade por Classe

Para cada feature no conjunto final, plotar curvas KDE sobrepostas para as 5 superclasses (NORM, MI, CD, HYP, STTC) usando apenas amostras com label único (1 superclasse). Organizar em grid de `N_features` plots (usar subplots compactos com `figsize` proporcional). Colorir cada classe consistentemente ao longo de todo o notebook e do relatório.

Para cada feature, calcular o **overlap index** entre distribuições: para duas classes A e B, $\text{overlap} = \int \min(\text{KDE}_A(x), \text{KDE}_B(x)) dx$. $\text{Overlap} = 0 \rightarrow$ classes completamente separáveis por esta feature; $\text{overlap} = 1 \rightarrow$ distribuições idênticas. Calcular a matriz de overlap 5×5 por feature e apresentar como heatmap.

Subseção 2.2 — Análise Discriminante Linear (LDA)

Aplicar `sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver='svd')` nos dados de treino com labels de superclasse (usando apenas registros de label único para a análise de separabilidade). LDA encontra projeções que maximizam a razão de variância entre classes / variância intra-classes (critério de Fisher).

Com 5 classes, LDA produz no máximo 4 componentes discriminantes. Plotar: - Scatter plot dos 2 primeiros componentes LDA (LD1 vs. LD2) com pontos coloridos por classe e elipses de confiança (95%) - Gráfico de barras com a variância discriminante explicada por cada LD

Comparar com o scatter plot do PCA do Entregável 8 — LDA deve mostrar melhor separação por ser supervisionado.

Subseção 2.3 — Índices de Separabilidade por Par de Classes

Para cada par de superclasses (10 pares no total), calcular a distância de Mahalanobis entre centroides: $D^2 = (\mu_A - \mu_B)^T S_{\text{pooled}}^{-1} (\mu_A - \mu_B)$ onde S_{pooled} é a matriz de covariância ponderada das duas classes.

Distâncias Mahalanobis > 3 indicam boa separabilidade; < 1 indica sobreposição substancial. Apresentar matriz 5×5 das distâncias. Identificar os pares mais difíceis de separar (menor distância) — estes serão os casos mais desafiadores para os algoritmos de RP.

Seção 3 — Análise de Balanceamento das Classes

Subseção 3.1 — Distribuição Atual das Classes

Calcular o número de amostras (janelas) por superclasse no dataset final, separado por split (treino/val/teste). Apresentar em tabela e gráfico de barras. Calcular o Imbalance Ratio (IR): $IR = n_{\text{classe_maior}} / n_{\text{classe_menor}}$. $IR > 3$ é considerado desbalanceamento moderado; $IR > 10$ é severo.

Calcular também o percentual de amostras multi-label (janelas pertencentes a mais de uma superclasse).

Subseção 3.2 — Distribuição por Fold

Para cada fold (1–10), criar tabela com a contagem de amostras por superclasse. Verificar que a estratificação está funcionando: as distribuições dos folds devem ser aproximadamente similares entre si. Calcular a variância do percentual de cada classe ao longo dos 10 folds — deve ser baixa ($< 5\%$).

Subseção 3.3 — Estratégias de Balanceamento Recomendadas para o RP

Escrever uma seção de recomendações formais (não implementar — são para o módulo de RP):

- **Class weights:** `class_weight='balanced'` disponível em SVM, Logistic Regression e Random Forest do sklearn. Multiplica o peso do erro para amostras de classes minoritárias. Recomendado como abordagem base.
- **SMOTE (Synthetic Minority Oversampling Technique):** gera amostras sintéticas interpolando entre amostras reais da classe minoritária no espaço de features. Implementar com `imbalanced-learn`. Aplicar **apenas no conjunto de treino** (nunca no val/teste).
- **Undersampling aleatório:** remover amostras da(s) classe(s) majoritária(s) até atingir proporção desejada. Perda de informação — menos recomendado.
- **Métricas de avaliação:** usar **Macro F1-score** (média não ponderada do F1 por classe — trata classes igualmente), **Balanced Accuracy** (média das recalls por classe) e **Macro AUROC** (não requer threshold).

Evitar accuracy simples e micro-averaged metrics — favoreceriam a classe NORM por ser a mais frequente.

Seção 4 — Inspeção Final de Integridade

Subseção 4.1 — Verificação Formal de Data Leakage

Para cada transformação aplicada, verificar que foi ajustada apenas nos folds 1–8:

Transformação	Ajustada em	Aplicada em	Arquivo de parâmetros
Winsorização	folds 1–8	todos	preprocessing_params.pkl
RobustScaler	folds 1–8	todos	scaler_params.pkl
PCA	folds 1–8	todos	pca_model.pkl
Imputação de NaN	folds 1–8	todos	preprocessing_params.pkl

Verificação programática: carregar cada arquivo `.pkl`, verificar que foi ajustado sobre `X_treino` (checar shape do conjunto de ajuste), e que as transformações dos folds 9 e 10 usam os parâmetros de treino (não recalculam).

Subseção 4.2 — Verificação de Contaminação de Features

Listar todas as colunas no `features_final.parquet`. Verificar que **não existe** nenhuma das seguintes: `scp_codes` (o alvo), `report` (contém o diagnóstico textual), `heart_axis` (informação diagnóstica), `infarction_stadium1/2` (informação diagnóstica derivada do laudo). Verificar que os únicos labels são as colunas binárias `label_NORM`, `label_MI`, `label_CD`, `label_HYP`, `label_STTC` (e as colunas de identificação).

Subseção 4.3 — Verificação de Integridade do Fold de Paciente

Verificar que nenhum `patient_id` aparece em mais de um fold: `assert df.groupby('patient_id')['strat_fold'].nunique().max() == 1`. Esta é a verificação mais importante de data leakage no PTB-XL — um paciente com múltiplos registros não pode ter parte no treino e parte no teste.

Subseção 4.4 — Checklist Final

Item	Status	Observação
Normalidade verificada e documentada	/	(resultado do Entregável 3)
Homocedasticidade verificada e documentada	/	

Item	Status	Observação
Multicolinearidade controlada ($VIF < 5$)	/	n features removidas
Data leakage ausente (parâmetros do treino)	/	
Folds de paciente respeitados	/	
Labels propagados corretamente	/	
Registros de categoria SQI C excluídos	/	n rejeitados
Threshold de likelihood documentado (50%)	/	
Features interpretáveis e com justificativa	/	
Separabilidade das classes avaliada	/	par mais difícil = X vs. Y

Seção 5 — Ficha Técnica do Dataset Final

Subseção 5.1 — Sumário Completo

Criar tabela técnica final: - Número total de amostras (janelas): separado por treino/val/teste - Número de features: separado por domínio (tempo, frequência, tempo-frequência, não lineares) - Distribuição por superclasse: n amostras por classe por split - Imbalance Ratio: por split - Percentual multi-label: por split - Frequência de amostragem dos sinais originais - Tamanho da janela e overlap utilizados - Técnicas de pré-processamento aplicadas (lista) - Parâmetros dos filtros (fc, ordem, tipo) - VIF máximo no conjunto final - Separabilidade: distância Mahalanobis mínima entre pares de classes

Subseção 5.2 — Mapa Completo do Pipeline

Criar um diagrama textual (tabela markdown) com cada etapa do pipeline:

```

ENTRADA: ptbxl_database.csv + records100/
↓
[E1] Documentação de aquisição + enriquecimento de metadados
    → ptbxl_metadata_enriched.csv
↓
[E2] Cálculo de SQI + rejeição de registros de baixa qualidade
    → ptbxl_com_sqi.csv (+ rejected_ecg_ids.txt)
↓
[E3] Análise estatística (normalidade, homocedasticidade, correlação)
    → estatistica_inicial_resultados.csv

```

↓

[E4] Pipeline de limpeza: hp(0.5Hz) → lp(40Hz) → winsorização
→ sinais_limpos_100hz + preprocessing_params.pkl

↓

[E5] Segmentação: janelas 2.5s overlap 50% + beat segmentation
→ janelas_segmentadas.npy + batimentos.npy

↓

[E6] Extração de features: tempo + frequência + wavelet + não-lineares
→ features_raw.parquet (N_janelas × ~300 features)

↓

[E7] Engenharia: razões + Δ + normalização robusta (treino only)
→ features_engineered.parquet + scaler_params.pkl

↓

[E8] PCA: 95% variância retida
→ features_pca.parquet + pca_model.pkl

↓

[E9] Seleção: Filter(ANOVA+MI+ReliefF) + Wrapper(SFS) + Embedded(LASSO)
→ features_final.parquet + feature_selection_report.csv

↓

[E10] Validação: VIF + separabilidade + balanceamento + checklist
→ dataset_final_X.parquet + dataset_final_y.parquet + folds_assignment.csv + pipeline_params.pkl

↓

SAÍDA: Dataset pronto para SVM, kNN, RF, NN, CNN

Seção 6 — Handover Formal para o Módulo de RP

Escrever texto declarando formalmente que o dataset está pronto. Listar os arquivos entregues com descrição de cada um:

- `dataset_final_X.parquet` — matriz de features (shape: N_amostras × N_features), com colunas `janela_id`, `ecg_id`, `patient_id`, `strat_fold` como índice, e todas as features numéricas normalizadas como colunas de dados
- `dataset_final_y.parquet` — matriz de labels (shape: N_amostras × 5), com colunas binárias `label_NORM`, `label_MI`, `label_CD`, `label_HYP`, `label_STTC`
- `folds_assignment.csv` — mapeamento de `ecg_id` para `strat_fold` e `split` (train/val/test)
- `pipeline_params.pkl` — dicionário Python serializado contendo todos os parâmetros aprendidos nos dados de treino: limiares de winsorização, parâmetros do scaler, modelo PCA, features selecionadas, VIF por feature

Lista de recomendações formais para o RP: 1. Usar `strat_fold` para validação cruzada — nunca shufflear sem respeitar `patient_id` 2. Métricas recomendadas: Macro F1, Balanced Accuracy, Macro AUROC 3. Balanceamento: iniciar com

`class_weight='balanced'`; se insuficiente, aplicar SMOTE no treino 4. Para modelos baseados em distância (kNN, SVM RBF): usar `features_pca.parquet` (dados reduzidos); para modelos baseados em árvore (RF, XGBoost): usar `features_final.parquet` (features originais — árvores não sofrem da maldição da dimensionalidade) 5. Para deep learning (CNN): usar os sinais brutos segmentados do Entregável 5 diretamente; não usar features

ENTREGÁVEL FINAL INTEGRADOR — Relatório Técnico-Científico

Formato: LaTeX com template TCC UFC; Overleaf

Objetivo: Documento único coeso que integra todos os 10 entregáveis em uma narrativa científica completa. Cada seção do relatório corresponde a um entregável, mas é escrita como um documento científico — não como uma coleção de notebooks.

Estrutura Detalhada do Documento

Capa Título: “Desenvolvimento de um Pipeline de Processamento de Biossinais para Reconhecimento de Padrões em Eletrocardiograma Clínico: Aplicação ao Dataset PTB-XL”. Autores, disciplina, orientador, instituição (UFC), semestre.

Resumo (Abstract) 250–300 palavras. Conter obrigatoriamente: contexto clínico (diagnóstico cardiovascular por ECG), dataset utilizado (PTB-XL: 21.837 registros, 18.885 pacientes, 12 derivações, labels SCP-ECG), objetivo (pipeline de processamento para RP), metodologia em alto nível (10 etapas: aquisição → SQI → estatística → limpeza → segmentação → features → engenharia → PCA → seleção → validação), principais resultados quantitativos (`n_features` finais, `n_amstras`, variância PCA retida, maior VIF, melhor separabilidade), conclusão (dataset pronto para RP com `X` features). Escrever em português e inglês.

1. Introdução

1.1 Contextualização do Problema Descrever o ECG como ferramenta diagnóstica não invasiva para doenças cardiovasculares. Motivar o uso de aprendizado de máquina para automação da interpretação de ECG: volume crescente de exames, escassez de cardiologistas em regiões remotas, auxílio à decisão clínica. Citar o estado da arte em classificação automática de ECG (algoritmos com desempenho comparável a cardiologistas foram publicados).

1.2 Desafios e Motivação Identificar os dois principais obstáculos para o avanço no campo: (a) ausência de datasets públicos grandes e bem anotados, e (b)

ausência de protocolos de avaliação padronizados. Motivar a escolha do PTB-XL como solução para (a).

1.3 Objetivos Objetivo geral: desenvolver e documentar um pipeline completo e reproduzível de processamento de sinais de ECG para preparação de dados para Reconhecimento de Padrões. Objetivos específicos: implementar avaliação de qualidade de sinal, limpeza por filtragem, segmentação temporal, extração multi-domínio de features, engenharia de features, redução de dimensionalidade, seleção de atributos e validação estatística final.

1.4 Organização do Documento Descrição breve de cada seção.

2. Fundamentação Teórica

2.1 O Sinal de ECG: Origem e Características Fisiologia cardíaca básica: despolarização e repolarização, ondas P-QRS-T, intervalos PR e QT, segmento ST. Diagrama das 12 derivações e o que cada uma “vê”. Características temporais e espectrais do ECG normal: banda 0,05–150 Hz, energia concentrada no complexo QRS (5–40 Hz). Relevância clínica de cada componente morfológico: onda P para avaliação atrial, QRS para ventricular, ST para isquemia.

2.2 Classificação de Diagnósticos ECG pelo Padrão SCP-ECG Explicar o padrão SCP-ECG: 71 statements cobrindo diagnóstico, forma e ritmo. Hierarquia de superclasses e subclasses (descrever as 5 superclasses com exemplos de diagnósticos em cada uma). Sistema de verossimilhança (likelihoods de 0–100%) e sua interpretação.

2.3 Qualidade de Sinal em ECG Tipos de ruído em ECG: interferência da rede elétrica (50/60 Hz), ruído muscular (EMG, 0–500 Hz), drift de baseline (< 0,5 Hz), artefatos de movimento, ruído de eletrodo. Técnicas de quantificação: SNR, kurtosis, entropia espectral. Relevância da qualidade do sinal para a validade dos diagnósticos.

2.4 Pipeline de Processamento de Biossinais Fluxo geral: aquisição → qualidade → limpeza → segmentação → extração de features → seleção → classificação. Justificativa da ordem.

2.5 Extração de Features Multi-Domínio para ECG Subseção por domínio (tempo, frequência, tempo-frequência, não lineares), com definição matemática formal de cada feature relevante e referências bibliográficas. Fórmulas devem ser apresentadas em LaTeX com notação consistente.

2.6 Redução de Dimensionalidade e Seleção de Atributos PCA: formulação matemática, eigenvectors e eigenvalues, variância explicada. Seleção filter/wrapper/embedded: definição formal e trade-offs. Correção de múltiplas comparações: problema de familywise error rate, Bonferroni e FDR.

3. Materiais e Métodos

3.1 Dataset PTB-XL Descrever completamente o dataset: origem, coleta, anonimização, estrutura de arquivos, metadados disponíveis, distribuição demográfica

(tabela), distribuição de labels (tabela das 5 superclasses), processo de anotação, verossimilhança dos diagnósticos, estrutura de folds. Justificar a escolha do dataset para o presente trabalho.

3.2 Avaliação de Qualidade do Sinal Descrever as 5 métricas de SQI implementadas (com fórmulas), o score composto, os limiares e as categorias A/B/C.

3.3 Pipeline de Pré-Processamento Descrever cada filtro aplicado: tipo (Butterworth/IIR notch), ordem, frequências de corte, método de aplicação (filtfilt para fase zero), justificativa. Descrever winsorização: percentis, base de cálculo (folds de treino), interpretação.

3.4 Estratégia de Segmentação Descrever a Opção B (janelas fixas) e a Opção C (beat segmentation): parâmetros, algoritmo de detecção de pico R, propagação de labels. Justificar os parâmetros escolhidos com base na fisiologia cardíaca.

3.5 Extração de Features Tabela completa: nome da feature, domínio, derivação(ões), fórmula ou referência, interpretação fisiológica.

3.6 Engenharia de Features Descrever features derivadas criadas, normalização aplicada, estratégia anti-data-leakage.

3.7 Redução de Dimensionalidade Parâmetros do PCA: critério de escolha de componentes (95% de variância), análise de loadings.

3.8 Seleção de Atributos Três métodos aplicados, critério de combinação, validação estatística com correção FDR.

3.9 Validação Final Análise VIF, separabilidade (Mahalanobis), balanceamento de classes.

4. Resultados

4.1 Avaliação de Qualidade do Sinal Tabela de distribuição SQI (A, B, C) por split e por superclasse. Figura comparativa de segmentos bons vs. ruins. Concordância com metadados originais do dataset.

4.2 Análise Estatística Inicial Tabela de estatística descritiva das variáveis principais. Tabela de resultados dos testes de normalidade. Tabela de homocedasticidade. Heatmap de correlação de Spearman.

4.3 Efeito do Pré-Processamento Tabela de Cohen's d (antes vs. depois) por grupo (Limpo, Ruído 50Hz, Baseline) e por métrica. Figura de espectros antes e depois.

4.4 Segmentação Tabela de janelas geradas por superclasse. Exemplos de janelas por classe.

4.5 Features Extraídas Tabela resumo de features por domínio. Figuras representativas de cada domínio.

4.6 Redução de Dimensionalidade Scree plot. Figura de projeção nos 2 primeiros PCs por superclasse. Análise de loadings (tabela).

4.7 Seleção de Atributos Tabela final de features selecionadas com todos os critérios. Figura de ranking de relevância.

4.8 Dataset Final Ficha técnica completa (tabela). Análise de separabilidade (scatter LDA, matriz Mahalanobis). Checklist de integridade.

5. Discussão

5.1 Qualidade dos Dados vs. Desempenho Esperado Discutir o impacto dos 22,99% de registros com problemas de qualidade. Como a filtragem (E4) mitigou os problemas. O que permanece como limitação.

5.2 Adequação do Pipeline ao Contexto Multi-Label Discutir os desafios específicos do problema multi-label do PTB-XL: propagação de labels, seleção de features para múltiplas classes, balanceamento em cenário multi-label. O que o pipeline fez bem e o que poderia ser melhorado.

5.3 Features Selecionadas e Sua Relevância Clínica Analisar criticamente o conjunto de features final: quais dominam? Correspondem ao que a literatura clínica identifica como importantes para ECG? Há features inesperadamente relevantes ou irrelevantes?

5.4 Limitações do Pipeline Mencionar: (a) janelas de 10s são curtas para análise robusta de HRV; (b) a segmentação por janelas fixas pode fragmentar complexos QRS entre janelas adjacentes; (c) a ausência de anotações beat-by-beat limita a validação da morfologia individual; (d) o dataset é europeu (1989–1996) e pode não representar populações de outras etnias ou equipamentos modernos.

6. Conclusão

Síntese de cada etapa e seus resultados principais em um parágrafo por entregável. Afirmação de que o dataset está pronto para RP. Direcionamentos futuros: implementação de modelos de RP, análise de features de deep learning, validação cruzada completa, análise de robustez em sinais de baixa qualidade.

Referências Bibliográficas Usar BibTeX. Incluir obrigatoriamente: - Wagner et al. (2020) — artigo de descrição do PTB-XL (publicado na *Scientific Data*, Nature) - Task Force of the European Society of Cardiology (1996) — guidelines de HRV (Circulation) — referência padrão para features de HRV - Pan & Tompkins (1985) — algoritmo de detecção de pico R (IEEE Transactions on Biomedical Engineering) - Sechidis et al. (2011) — estratificação multi-label (Machine Learning and Knowledge Discovery in Databases) — base do método de folds do PTB-XL - Kohavi (1995) — validação cruzada e bootstrap (IJCAI) - Referências para cada feature não-linear: Richman & Moorman (2000) para SampEn; Peng et al. (1995) para DFA; Higuchi (1988) para dimensão fractal

Apêndices

Apêndice A — Tabela Completa de Features Nome, fórmula, domínio, derivação, range esperado, interpretação clínica. Uma linha por feature.

Apêndice B — Parâmetros dos Filtros Aplicados Tabela: filtro, tipo, ordem, frequência de corte, método de aplicação, justificativa.

Apêndice C — Tabela VIF Completa VIF por feature antes e depois da remoção iterativa.

Apêndice D — Código de Carregamento Mínimo Código Python de ~20 linhas que demonstra como carregar o dataset final, aplicar os parâmetros de pipeline salvos, e obter X_treino, y_treino, X_teste, y_teste prontos para um classificador sklearn. Isso garante reprodutibilidade completa.

Apêndice E — Distribuição Completa de Labels por Fold Tabela: fold $\times$ super-classe, com contagens e percentuais. Evidência da estratificação eficaz.

Fim do Guia — Versão 2.0 — Pipeline de Biossinais PTB-XL Guia completo e autossuficiente para implementação de nível TCC